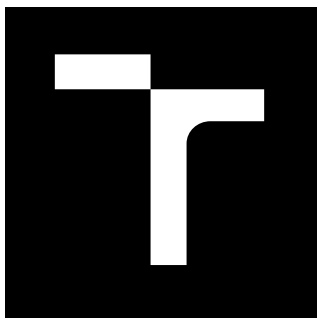




VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

ZVUKOVÝ EFEKT PRO SIMULACI KYTAROVÉHO KOMBA

AUDIO EFFECT FOR GUITAR COMBO SIMULATION

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Bartoloměj Pecháček

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. David Leitgeb

BRNO 2025



Bakalářská práce

bakalářský studijní program **Audio inženýrství**
specializace Zvuková produkce a nahrávání
Ústav telekomunikací

Student: Bartoloměj Pecháček

ID: 240191

Ročník: 3

Akademický rok: 2024/25

NÁZEV TÉMATU:

Zvukový efekt pro simulaci kytarového komba

POKYNY PRO VYPRACOVÁNÍ:

Navrhněte zvukový efekt simulující kytarový zesilovač a kytarový kabinet. K simulaci kytarového kabinetu použijte konvoluci zvukového signálu s impulsovou odezvou kabinetu. Efekt bude umožňovat výběr kabinetu, snímacího mikrofону a jeho polohy. Simulace kabinetu bude umožňovat také interpolaci mezi změřenými polohami mikrofónu. Dále bude efekt obsahovat simulaci předzesilovače s více režimy zkreslení a kmitočtový korektor s vlastnostmi typickými pro kmitočtové korektory kytarových zesilovačů. Efekt nejprve implementujte a otestujte v prostředí Matlab jako funkce generující zvukové soubory. Otestovaný a odladěný algoritmus implementujte v jazyce C++ jako VST plug-in modul s využitím frameworku JUCE.

DOPORUČENÁ LITERATURA:

- [1] ZÖLZER, Udo. DAFX: digital audio effects. 2nd ed. Chichester: Wiley, 2011. ISBN 978-0-470-66599-2.
[2] REISS, Joshua D. a MCPHERSON, Andrew P. Audio effects: theory, implementation and application. Boca Raton: CRC Press, 2014. ISBN 978-1-4665-6028-4.

Termín zadání: 10.2.2025

Termín odevzdání: 3.6.2025

Vedoucí práce: Ing. David Leitgeb

doc. Ing. Jiří Schimmel, Ph.D.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

ABSTRAKT

Práce se zabývá tematikou digitálního zpracování signálů, simulací kytarového zesilovače, konvolucí a jejím využití ve zpracování v reálném čase. Praktickým výstupem je VST3 plugin - simulátor kytarového zesilovače se třemi módy zkreslení, korekčním zesilovačem a simulací kabinetu s využitím naměřených impulsních odezev reálných reproduktorů.

KLÍČOVÁ SLOVA

C++, impulsní odezva, JUCE, konvoluce, korekční zesilovač, kytarový zesilovač, MATLAB, segmentovaná konvoluce, VST modul, zpracování v reálném čase

ABSTRACT

This thesis deals with digital signal processing, simulation of guitar amplifier, convolution and its usage in real-time processing. Practical outcome is a VST3 plugin - guitar amplifier simulator with three modes of distortion, tone stack and cabinet simulation using measured impulse responses of real speakers.

KEYWORDS

C++, convolution, guitar amplifier, impulse response, JUCE, MATLAB, real-time processing, segmented convolution, tone stack, VST plugin

PECHÁČEK, Bartoloměj. *Zvukový efekt pro simulaci kytarového komba*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2025. Vedoucí práce: Ing. David Leitgeb

Prohlášení autora o původnosti díla

Jméno a příjmení autora: Bartoloměj Pecháček
VUT ID autora: 240191
Typ práce: Bakalářská práce
Akademický rok: 2024/25
Téma závěrečné práce: Zvukový efekt pro simulaci kytarového komba

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno

.....

podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Davidu Leitgebovi za odborné vedení, konzultace, trpělivost a podnětné návrhy k práci.

Obsah

Úvod	10
1 Teoretická část studentské práce	11
1.1 Kytarové kombo	11
1.1.1 Elektronka	11
1.1.2 Předzesilovač	12
1.1.3 Digitální emulace předzesilovače	15
1.1.4 Korekční zesilovač	15
1.2 Konvoluce	16
1.2.1 Diskrétní konvoluce	16
1.2.2 Lineární konvoluce	17
1.2.3 Kruhová konvoluce	17
1.2.4 Rychlá konvoluce	17
1.3 Konvoluce v reálném čase	19
1.3.1 Segmentovaná konvoluce	19
2 Výsledky studentské práce	23
2.1 Metodika měření	23
2.2 Výsledky měření	23
2.3 Implementace pomocí MATLAB App Designer	25
2.3.1 Rozbor jednotlivých funkcí	26
2.4 Implementace v MATLAB knihovně Audio Toolbox	27
2.4.1 Zkreslení	28
2.4.2 Kmitočtový korektor	28
2.5 Implementace v C++ pomocí JUCE	29
2.5.1 Zpracování signálu	29
2.5.2 Vstupní a výstupní zesílení	30
2.5.3 Zkreslení	30
2.5.4 Korekční zesilovač	33
2.5.5 Konvoluce	33
2.6 Uživatelské rozhraní	34
2.6.1 Instalace	36
2.7 Testování VST3 modulu	36
2.7.1 Analýza zkreslení	36
2.7.2 Odezva filtrů korektoru	39
Závěr	40

Literatura	41
Seznam symbolů a zkratk	45
Seznam příloh	46
A Zdrojové soubory MATLAB	47
B Soubory JUCE	48

Seznam obrázků

1.1	Nelineární závislost proudu anodou (I_a) na napětí na mřížce (V_g) při odečítání hodnot ze zatěžovací přímky. [3]	12
1.2	Výstupní signál elektronky při překročení obou limitů napětí mřížky [2].	14
1.3	Diagram rychlé lineární konvoluce s využitím kruhové konvoluce v transformované doméně, podle Stockhama. [21]	18
1.4	Schéma metody Overlap-Add [24].	20
1.5	Schéma metody Overlap-Save [23].	20
1.6	Schéma standardního algoritmu se segmentací o konstantní velikosti.[19]	22
2.1	Měřicí ústrojí.	23
2.2	Porovnání frekvenčních charakteristik měřených reproduktorů: (a) Marshall 4x12 kabinet, (b) Marshall JCM200 - DSL401, (c) Line6 Spider Valve 112. Pozice mikrofону konstantní ($x = 0$ cm, $y = 0$ cm).	24
2.3	Frekvenční charakteristiky v pozicích: (a) $x = 0$ cm, (b) $x = 5$ cm, (c) $x = 10$ cm. Marshall kabinet, vzdálenost mikrofону konstantní ($y = 0$ cm).	25
2.4	Frekvenční charakteristiky v pozicích: (a) $y = 0$ cm, (b) $y = 10$ cm, (c) $y = 40$ cm. Marshall kabinet, pozice v ose x konstantní ($x = 0$ cm).	25
2.5	Uživatelské rozhraní MATLAB aplikace.	26
2.6	Blokové schéma modulu.	30
2.7	Přenosové funkce Polettiho algoritmu.	32
2.8	Přenosové funkce využité ve Wavefolder typu zkreslení.	33
2.9	Grafické rozhraní modulu VST3.	35
2.10	Frekvenční spektra tvořená algoritmem podle Yamahy (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.	37
2.11	Spektra tvořená algoritmem podle Polettiho (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.	38
2.12	Spektra tvořená algoritmem Wavefolder (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.	38
2.13	Odezvy korekčního zesilovače v neutrálním stavu (a) a pro různá nastavení jednotlivých enkodérů (b–d).	39

Úvod

Tato práce se věnuje oblasti digitálního zpracování audio signálu, pojednává převážně o algoritmech konvoluce v reálném čase, jejich využití pro simulaci kytarových kabinetů a emulaci elektronkového kytarového zesilovače.

Cílem práce bylo navrhnout simulátor kytarového zesilovače v prostředí aplikace MATLAB pro otestování algoritmů zpracování signálu a následně vytvořit VST3 modul stejné funkčnosti za pomoci frameworku JUCE pro C++.

Teoretická část práce přiblíží funkci elektronkového zesilovače a možnosti jeho digitální emulace. Dále je rozebrána konvoluce a algoritmy pro její výpočet v reálném čase s využitím rychlé Fourierovy transformace.

V praktické části jsou rozebrány výsledné programy z prostředí MATLAB i VST3 modul, jejich vnitřní struktura a jsou zde rozepsány procesy jednotlivých funkcí. Na závěr je analyzován výstup VST3 modulu.

1 Teoretická část studentské práce

V této kapitole jsou rozebrány teoretické znalosti, které jsou využity v praktické části pro realizaci jednotlivých algoritmů, ze kterých se skládá výsledný modul.

1.1 Kytarové kombo

Kytarovým kombem rozumíme spojení kytarového zesilovače s reproduktorem umístěným v ozvučnici do jednoho objektu. Zesilovač se zpravidla sestává z předzesilovače, kmitočtového korektoru, koncového zesilovače a transformátoru, který je napojen na reproduktor [1].

1.1.1 Elektronika

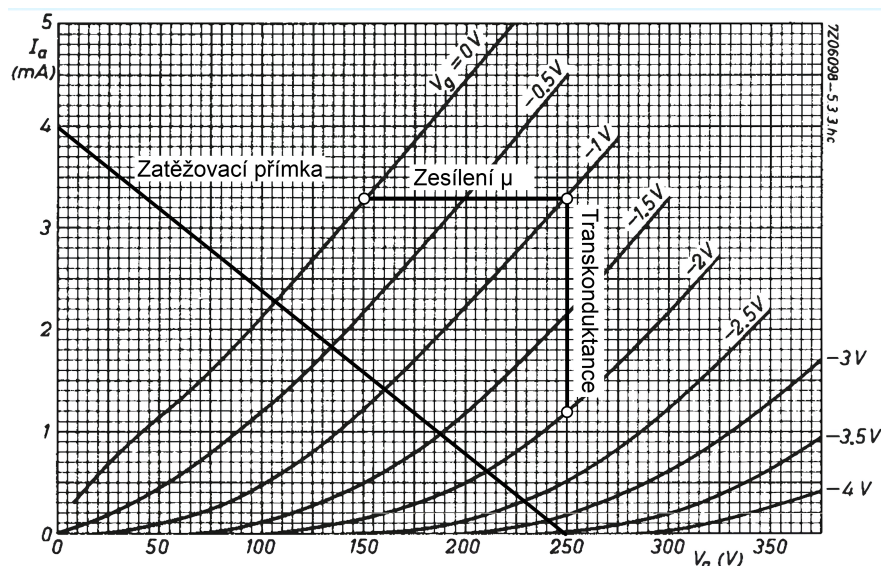
Jelikož se práce zabývá emulací elektronkového předzesilovače, je nutné nejprve stručně popsat vlastnosti elektronek.

Dioda

Základní variantou je dioda složená z nahřívané katody, která vlivem Edisonova jevu emituje elektrony a anody, na kterou je přivedeno kladné napětí, přitahuje volné elektrony a vzniká tedy proud [2]. Při přivedení střídavého proudu dochází k přenosu pouze kladné poloviny signálu.

Trioda

Trioda je diodou s přidanou mřížkou G_1 mezi elektrodami. Pokud mřížka není napájena, nijak neovlivňuje tok elektronů, avšak pokud je napájena záporným napětím, tok elektronů omezuje a pokud je napětí dostatečně vysoké, tok elektronů úplně zastaví. Napětí na mřížce může být násobně menší než napětí na elektrodách - toho je využito při zesilování signálů. Mřížka napájena vstupním signálem způsobí promítnutí tvaru signálu do proudu mezi elektrodami [2]. Aby trioda přenášela kladnou i zápornou polovinu vstupního signálu, musí být signál posunutý tak, aby nedosahoval kladného napětí. Toho lze docílit například přidáním rezistoru na katodu [2]. Dochází však k asymetrickému zesílení mezi kladnou a zápornou půlvlnou a to z důvodu nelineárního vztahu napětí na mřížce a proudu anodou. Toto je ilustrováno na obr. 1.1.



Obr. 1.1: Nelineární závislost proudu anodou (I_a) na napětí na mřížce (V_g) při odečítání hodnot ze zatěžovací přímky. [3]

Tetroda a pentoda

Přidáním druhé mřížky G_2 do triody, s klidovým napětím blízkým napětí anody, lze potlačit parazitní kapacitní zpětnou vazbu mezi mřížkou G_1 a anodou [4]. Dochází však k sekundární emisi, kdy elektrony s příliš vysokou energií vyrazí elektrony z materiálu anody [2]. Důsledkem je část přenosové charakteristiky, kdy zvyšování napětí na anodě vede ke snížení proudu anodou, tohoto využívá například dynatronový oscilátor [5].

Pentoda je rozšířena o třetí mřížku G_3 s napětím blízkým napětí katody, která zpomaluje elektrony urychlené mřížkou G_2 a odpuzuje elektrony vyražené sekundární emisí [6]. Mívají vyšší vnitřní odpor a zkreslení [4].

1.1.2 Předzesilovač

Předzesilovač zesiluje vstupní slabý signál na úroveň požadovanou na vstupu koncového (výkonového) zesilovače [7]. Zároveň slouží k impedančnímu přizpůsobení – kytarové magnetické snímače mívají impedanci v řádech $k\Omega$ [8]. Samotné zesílení zajišťují elektronky nebo tranzistory [1], tato práce se však zaměřuje pouze na emulaci elektronky. Pokud je zesilovací komponent přetížen, signál je obohacen o další harmonické složky, což je v tomto případě žádoucí [9].

Na základě přenosové charakteristiky či principu činnosti lze zesilovače dělit do tříd [7], v textu jsou uvedeny pouze třídy relevantní k tématu této práce.

Třída A

Zesilovače třídy A využívají jednu elektronku (či tranzistor) pro zesílení celého vstupního signálu. Elektronkou protéká nenulový klidový proud, který ji udržuje ve vodivém stavu [7]. Pokud je klidový proud správně nastaven, produkují velmi malé zkreslení [4], jinak může docházet k asymetrickému zkreslení. Tato nelinearita je popsána exponenciální funkcí:

$$I_a = k(V_a + \mu V_g)^{3/2}, \quad (1.1)$$

kde I_a a V_a je proud, respektive napětí na anodě, V_g je napětí mřížky, μ je zesílení a k je konstanta závislá na typu elektronky [2]. Zesílení μ je vypočteno ze vztahu mezi vstupním a výstupním napětím $\Delta V_g / \Delta V_a$, nebo lze graficky odečíst ze specifikací elektronky, zaznačeno na obr. 1.1. Příkladem je předzesilovač Gibson GA-5, který sestává ze dvou zesilovačů zapojených v sérii využívajících triodu 12AX7 [2].

Vyššího výkonu je možné dosáhnout pomocí zapojení push-pull, kdy je vstupní signál rozdělen do dvou větví, přičemž v jedné z nich je fáze signálu obrácena o 180°. Oproti normálnímu zapojení, kde je zátěž uzemněna, je nyní zátěž připojena pouze na výstup zesilovače, který produkuje identické signály s opačnou fází. Napětí na zátěži je dvakrát větší, výkon tedy čtyřnásobný [2].

Třída B

Třída B je specifikována nulovým klidovým proudem anodou. Poloha pracovního bodu tedy způsobí přenos pouze jedné půlvlny vstupního signálu, umožňuje však vyšší výkony a vyšší účinnost [4]. Při přechodu mezi vodivým a nevodivým stavem však vzniká nelineární přechodné zkreslení [7]. Vždy je využito zapojení push-pull, kdy je pár elektronek napájen vstupním signálem s navzájem opačnou fází. Výstup elektronek je efektivně sečten transformátorem, na který je napojena zátěž [2].

Přechodné zkreslení vzniká na obou polovinách signálu symetricky, při emulaci pomocí waveshaperu – objektu, který vstupní signál transformuje podle jeho přenosové funkce na výstupní [10] – je tedy vhodné volit symetrické přenosové funkce. Při použití tranzistorů namísto elektronek dochází v nízkých napětích k výpadku signálu, dokud napětí nedosáhne úrovně nutné pro spuštění vodivosti tranzistoru [2]. Tohoto je možné docílit i u elektronek jiným nastavením klidového proudu. Jeli-kož je signál symetricky měkce omezen, amplituda vzniklých lichých harmonických složek je vyšší než sudých [2].

Třída AB

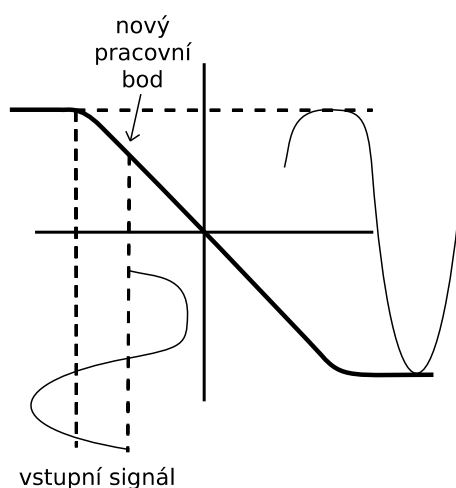
Je zaveden nízký klidový proud, elektronka tedy přenáší i část záporné poloviny vstupního signálu. Stejně jako u třídy B jsou dvě elektronky zapojeny jako push-

pull. Přechodné zkreslení je zmenšeno. U nízkých úrovní vstupního signálu zesilovač pracuje ve třídě A, u vyšších ve třídě B [7].

Přetížení mřížky

Pro získání většího zkreslení je možné mřížku úmyslně přetížit vyšším napětím, než je specifikováno. Frekvenční spektrum výsledného signálu závisí na stavbě zesilovače. U kaskády triod pracujících jako třída A vzniká primárně zkreslení bohaté na sudé harmonické, u zapojení třídy B vzniká více lichých harmonických z důvodu symetrického zkreslení a zároveň je dynamika signálu komprimována [2]. Specifikace elektronky uvádí horní a dolní limit úrovně vstupního napětí. Chování elektronky silně závisí na tom, jaký limit je překročen a v jaké třídě pracuje:

- **Třída A:** Při překročení dolního limitu – napětí na mřížce jde do vyšších záporných napětí – dojde k zastavení toku elektronů. Špičky výstupního signálu přesahující tento limit jsou tedy měkce omezeny (v závislosti na přenosové charakteristice elektronky). Při překročení horního limitu – napětí na mřížce je kladné – se z mřížky stane prakticky druhá anoda, mezi katodou a mřížkou vzniká dioda, která omezuje signál na vstupu elektronky. Tím dojde k vytvoření záporné stejnosměrné složky, což způsobí posunutí úrovně předpětí a operačního bodu elektronky [2]. Výstupní signál je tedy asymetricky zesílen, dojde k omezení kladné půlvlny a „protažení“ záporné půlvlny. Toto je ilustrováno na obr. 1.2.
- **Třída B:** Zde dochází ke stejným jevům jako u třídy A, vliv na výstupní signál je však rozdílný. Špičky signálu jsou měkce omezeny, posun pracovního bodu však způsobí, že část signálu není zesílena. Vzniká zde vyšší přechodné zkreslení [2].



Obr. 1.2: Výstupní signál elektronky při překročení obou limitů napětí mřížky [2].

1.1.3 Digitální emulace předzesilovače

Způsobů jak emulovat nelineární přenos zesilovače je vícero, nejjednodušším je použití waveshaperu [11]. Jeho přenosová funkce udává, jaká je okamžitá výstupní úroveň na základě úrovně vstupní. To, jaké harmonické složky jsou generovány záleží na tvaru přenosové funkce a vstupním signálu. Pokud uvažujeme harmonický vstupní signál, potom lichá přenosová funkce generuje pouze liché harmonické, sudá pouze sudé harmonické a asymetrická funkce generuje všechny harmonické [12, 13]. Patenty, na jejichž základě byl vytvořen vlastní algoritmus v praktické části práce, využívají vícero bloků waveshaperů paralelně či v sérii [14, 15]. Mezi další možnosti emulace patří simulace analogových obvodů pomocí Wave Digital Filter či neuronových sítí [16].

1.1.4 Korekční zesilovač

Korekční zesilovač je v analogovém provedení obvykle sítí tří filtrů, které navzájem interagují a jejichž vlastnosti jsou složitě simulovatelné [17], například použitím Wave Digital filtrů. Pro účely této práce bylo zvoleno korekční zesilovač pouze emulovat pomocí digitálních filtrů. Použité typy filtrů jsou dále stručně diskutovány.

Digitální kmitočtové filtry

Kmitočtový filtr je lineární, časově neměnný (LTI) systém, který propouští část frekvenčního spektra a jinou potlačuje [18]. Na základě délky impulsní odezvy je lze dělit na filtry s nekonečnou odezvou (IIR), které jsou využity v této práci, a odezvou konečnou (FIR). Přenos kanonického filtru je určen jeho přenosovou funkcí [12]:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}, \quad (1.2)$$

kde a a b jsou koeficienty filtru. V tab. 1.1 je uveden pouze výpočet koeficientů peak filtru, jelikož koeficienty ostatních filtrů nejsou v práci vypočítány manuálně.

Mezi použité typy filtrů spadají [12]:

- Horní propust – je určena mezním kmitočtem, pod kterým je signál utlumen.
- Pásmová propust – je určena dvěma mezními kmitočty, mimo které je signál utlumen.
- Low shelf – je určen mezním kmitočtem a zesílením, které filtr aplikuje na signál pod mezním kmitočtem. Signál nad mezním kmitočtem zůstává nezměněn.
- High shelf – pracuje stejně jako low shelf, pouze aplikuje zesílení na signál nad mezním kmitočtem.
- Peak – aplikuje zesílení G na signál na středním kmitočtu f_c , ostatní části signálu jsou přeneseny beze změny.

Sklon přenosové charakteristiky filtru mezi přenosovým pásmem a utlumeným, v případě peak filtru šířka ovlivněného pásma, je určen kvalitou filtru Q [12].

	b_0	b_1	b_2	a_1	a_2
Zesílení	$\frac{1 + \frac{V_0}{Q} K + K^2}{1 + \frac{1}{Q} K + K^2}$	$\frac{2(K^2 - 1)}{1 + \frac{1}{Q} K + K^2}$	$\frac{1 - \frac{V_0}{Q} K + K^2}{1 + \frac{1}{Q} K + K^2}$	$\frac{2(K^2 - 1)}{1 + \frac{1}{Q} K + K^2}$	$\frac{1 - \frac{1}{Q} K + K^2}{1 + \frac{1}{Q} K + K^2}$
Zeslabení	$\frac{1 + \frac{1}{Q} K + K^2}{1 + \frac{1}{V_0 Q} K + K^2}$	$\frac{2(K^2 - 1)}{1 + \frac{1}{V_0 Q} K + K^2}$	$\frac{1 - \frac{1}{Q} K + K^2}{1 + \frac{1}{V_0 Q} K + K^2}$	$\frac{2(K^2 - 1)}{1 + \frac{1}{V_0 Q} K + K^2}$	$\frac{1 - \frac{1}{V_0 Q} K + K^2}{1 + \frac{1}{V_0 Q} K + K^2}$

Tab. 1.1: Vztahy pro výpočet koeficientů peak filtru, kde $K = \tan(\pi f_c / f_{vz})$ a $V_0 = 10^{G/20}$ [12].

1.2 Konvoluce

1.2.1 Diskrétní konvoluce

Diskrétní konvoluce je operace mezi dvěma funkcemi neurčité délky. Je popsána rovnicí [18]:

$$y(n) = x(n) * h(n) = \sum_{k=-\infty}^{\infty} x(k) \cdot h(n - k), \quad (1.3)$$

kde $x(n)$ a $h(n)$ jsou vstupní funkce, $y(n)$ je výsledná funkce taktéž s neurčitou délkou.

Pro daný LTI systém, který je popsán jeho impulsní odezvou $h(n)$, diskrétní konvoluce definuje signál na výstupu daného systému $y(n)$, pokud je na vstupu časově diskrétní signál $x(n)$.

Podle délky vstupních funkcí lze rozlišit dva případy [19]:

- Konvoluce dvou konečně dlouhých signálů. Tato operace je nejčastěji používána při offline zpracování audio signálu, například filtrace audio souboru filtrem s konečnou impulzní odezvou.
- Konvoluce nekonečně dlouhého signálu s konečně dlouhým signálem. Typické použití při filtraci potenciálně nekonečného vstupního signálu $x(n)$ s konečnou impulzní odezvou filtru $h(n)$ (FIR – Finite Impulse Response), kde výstupní signál $y(n)$ je kontinuálně dopočítáván s přísunem nových vstupních hodnot $x(n)$. Na počítačích je tohoto dosaženo výpočty po blocích o dané délce bloku B .

1.2.2 Lineární konvoluce

Lineární konvoluce potenciálně nekonečného vstupního signálu $x(n)$ s filtrem o délce N $h(n) = h(0), \dots, h(N-1)$ je definovaná jako:

$$y(n) = x(n) * h(n) = \sum_{k=0}^{N-1} x(n-k) \cdot h(k), \quad (1.4)$$

kde $y(n)$ je výsledný signál.

V případě že $x(n)$ má konečnou délku M a $h(n)$ je konečné délky N , je konvoluce limitovaná pro hodnoty k , kde dochází k překryvu posunutého signálu $x(n-k)$ s impulzní odezvou filtru $h(n)$ pro alespoň jeden prvek ($0 \leq n-k \leq M-1 \wedge 0 \leq k \leq N-1$). Mimo tento interval jsou hodnoty $x(n-k)$ a $y(k)$ nedefinované. Z toho plyne definice lineární konvoluce dvou konečně dlouhých funkcí [19]:

$$y(n) = x(n) * h(n) = \sum_{k=\max\{0,n\}}^{\min\{n-M+1, N-1\}} x(n-k) \cdot h(k), \quad (1.5)$$

kde výstupní signál $y(n)$ má délku $M+N-1$ [18].

1.2.3 Kruhová konvoluce

Na rozdíl od lineární konvoluce, která je aperiodická, kruhová konvoluce předpokládá periodicitu některého ze vstupních signálů. Je to z důvodu využití některé z diskrétních transformací pro efektivnější výpočet konvoluce, kde transformace předpokládá periodicitu vstupních funkcí. [19]

Vstupní signál $x(n)$ o délce $M \in \mathbb{N}$ periodizujeme na $\tilde{x}(n)$ s periodou M pomocí operace modulo: $\tilde{x}(n) = x(\text{mod}_M n)$ [20].

Kruhová konvoluce je poté definována jako [19]:

$$\tilde{y}(n) = x(n) \circledast h(n) = \sum_{k=0}^{M-1} \tilde{x}(n-k) \cdot \tilde{h}(k), \quad (1.6)$$

kde $\tilde{y}(n)$ je výsledný periodický signál s periodou M . Lze vybrat pouze jednu periodu pomocí některého z typů oken (např. pravoúhlé). Pro zamezení chyb by perioda $\tilde{x}(n)$ a $\tilde{h}(n)$ měla být stejná, proto se doplní signály nulami do požadované délky [18].

1.2.4 Rychlá konvoluce

Rychlá konvoluce je založená na transformování signálu do domény, kde je výpočet konvoluce méně náročný. Při použití diskrétní Fourierovy transformace (DFT) pomocí algoritmů rychlé Fourierovy transformace (FFT) lze dosáhnout rychlé kruhové konvoluce. Pokud budou vstupní signály vhodně doplněny nulami, je možná

i rychlá lineární konvoluce [19]. Tato metoda je dnes nejvíce rozšířená. DFT navíc disponuje vlastností, že kruhová konvoluce v časové doméně odpovídá vynásobení odpovídajících párů spektrálních koeficientů ve frekvenční doméně. Tato vlastnost je označována jako cyclic convolution property (CCP) [19].

FFT rychlá konvoluce

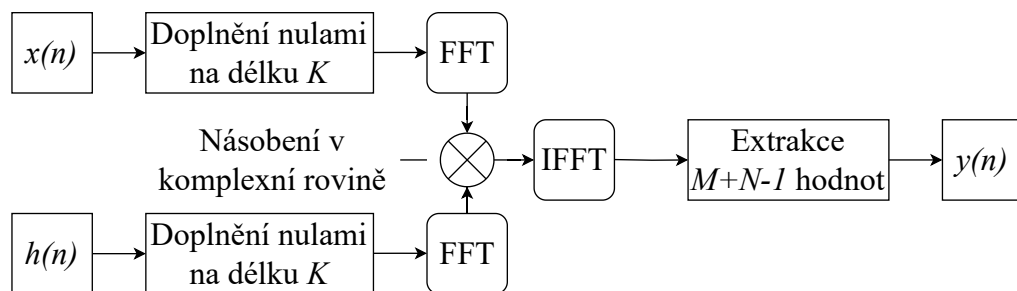
Mějme signály $x(n)$ o délce M a $h(n)$ o délce N . Lineární konvoluci $x(n) * h(n)$ je možné vypočítat v $\mathcal{O}(K \log K)$ pomocí rovnice:

$$\tilde{y}(n) = \mathcal{DFT}_{(N)}^{-1} \{ \mathcal{DFT}_{(N)} \{ x(n) \} \times \mathcal{DFT}_{(N)} \{ h(n) \} \}. \quad (1.7)$$

Pro dosažení správného výsledku je nutno použít FFT o velikosti K , kde K musí splňovat podmínku $K \geq M + N - 1$. Tato metoda navržená Stockhamem [21] je popsána na obr.1.3.

Oba vstupní signály jsou doplněny nulami na délku K a transformovány pomocí FFT o velikosti K . Výsledná spektra $X(k)$ a $H(k)$ o délce K , jsou K -krát párově vynásobena (v komplexní rovině). Tato operace je nazývána spektrální konvolucí. Výsledný signál je transformován zpět do časové domény zpětnou FFT, a jelikož platí podmínka $K \geq M + N - 1$ je vybráno prvních $M + N - 1$ hodnot.

Náročnost výpočtu tohoto algoritmu závisí na velikosti K , zda se mění filtr $h(n)$ či je jeho jednou vypočítané spektrum $H(k)$ možné použít pro více vstupních signálů, zda je vstupní signál reálný – spektrum DFT je poté symetrické a je nutné ukládat pouze polovinu hodnot. [19]



Obr. 1.3: Diagram rychlé lineární konvoluce s využitím kruhové konvoluce v transformované doméně, podle Stockhama. [21]

1.3 Konvoluce v reálném čase

1.3.1 Segmentovaná konvoluce

Základní premisou konvoluce po částech je rozdělení vstupního signálu a/nebo impulsní odezvy filtru na bloky, které jsou zpracovány zvlášť. Je nutné delší lineární konvoluce rozdělit na menší, aby byla zachována podmínka zpracování v reálném čase. V praxi je velikost bloku udávána nastavením vyrovnávací paměti zvukové karty či hostitelského softwaru. Algoritmus rychlé konvoluce lze aplikovat v reálném čase pomocí metod Overlap-Add (OLA) nebo Overlap-Save (OLS). Výpočet lze zrychlit rozdělením filtru na několik částí [19].

Předpokládejme vstupní signál o neznámé délce $x(n)$, který rozdělíme na bloky o velikosti B [19]:

$$x(n) = \sum_{i \geq 0} x_i(n - Bi), \text{ pokud platí } (0 \leq n < B, i \geq 0). \quad (1.8)$$

V případě kdy délka signálu $x(n)$ není dělitelná zvolenou velikostí bloku, je poslední blok doplněn nulami.

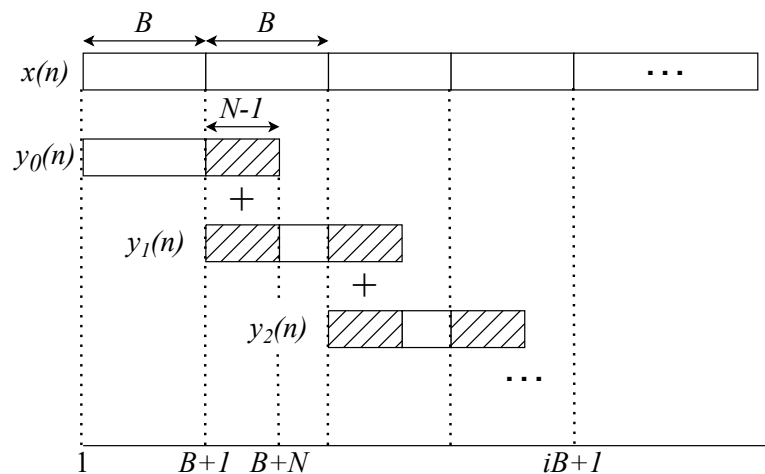
Segmentovaná konvoluce v reálném čase vždy využívá dělení na bloky konstantní velikosti, jelikož jsou vstupní hodnoty děleny na bloky s neměnnou velikostí již předem – opět zvukovou kartou či hostitelským softwarem. Zároveň to umožňuje využít jednou vypočtené frekvenční spektrum vstupního bloku pro konvoluci se všemi částmi filtru, čímž dojde ke zvýšení efektivity [19].

Metoda Overlap-Add

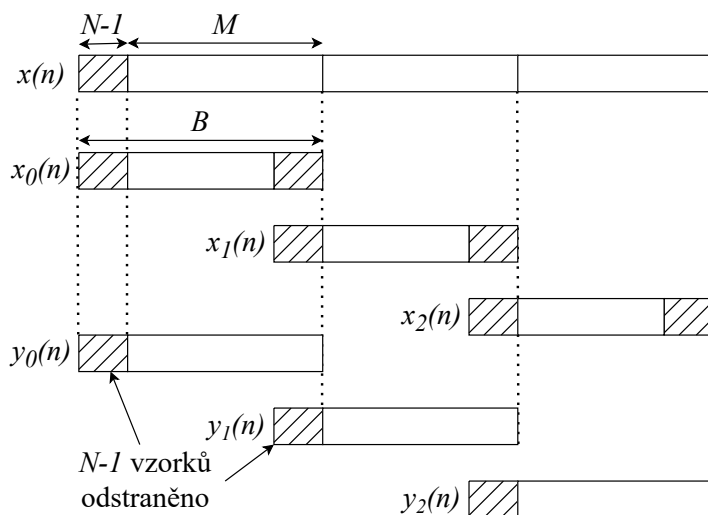
Při Overlap-Add metodě je vypočtena konvoluce každého bloku (velikosti B) vstupního signálu $x_i(n)$ s impulsní odezvou filtru $h(n)$ o délce N . Vzniklé částečné konvoluce $y_i(n)$ jsou delší než je délka bloku ($B + N - 1 > B$) a je nutné tedy překrývající se části $y_i(n)$ a $y_{i+1}(n)$ sečíst, viz obr. 1.4. [19] Tato metoda je využita v C++ implementaci efektu [22].

Metoda Overlap-Save

Před vstupní signál $x(n)$ se přidá $N - 1$ nul, následně se rozdělí na překrývající se bloky o velikosti $B = M + N - 1$ s délkou překryvu $N - 1$. Konvolucí $x_i(n) * h(n)$ vzniknou překrývající se bloky $y_i(n)$, kde je nutné prvních $N - 1$ hodnot odstranit. Výstupní signál $y(n)$ vznikne spojením bloků $y_i(n)$. Ilustrováno na obr. 1.5. [23]



Obr. 1.4: Schéma metody Overlap-Add [24].



Obr. 1.5: Schéma metody Overlap-Save [23].

Standardní algoritmus se segmentací o konstantní velikosti

Tato kapitola čerpá z disertační práce F. Wefers: Partitioned convolution algorithms for real-time auralization [19].

Tento algoritmus kombinuje OLS metodu s FFT konvolucí. Vstupní signál $x(n)$ je předáván po blocích o velikosti B (např. bloky vzorků ze zvukové karty), impulsní odezva filtru $h(n)$ je o velikosti N . Výsledný signál je vysílán na výstup taktéž po

blocích o velikosti B . Impulsní odezva filtru je rozdělena na P částí, podle

$$P = \lceil \frac{N}{L} \rceil, \quad (1.9)$$

kde L je délka části filtru. Jelikož je výsledek dělení zaokrouhlen, nemusí platit že $L \times P = N$. Podmínka která platí je $L \times P \geq N$, z toho vychází, že pro situace kde je $L \times P > N$ je nutné impulsní odezvu doplnit nulami. Každá část filtru musí být zpožděna o $n \times L$ hodnot, kde n je index dané části filtru.

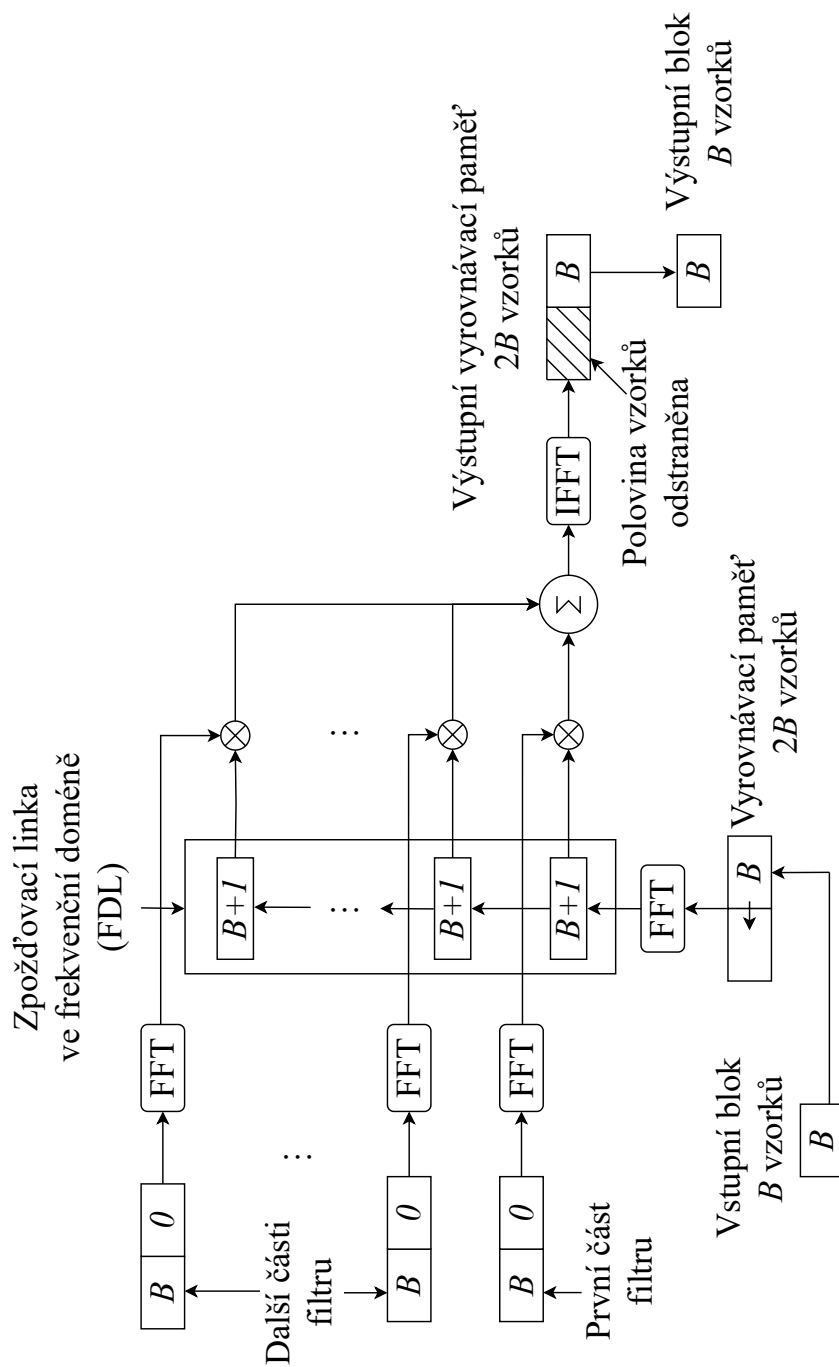
Velikost FFT transformace je pevně daná na $K = 2B$, z čehož vychází, že pro velikosti vstupních bloků B , které jsou nejčastěji mocniny dvou, je K také mocninou dvou a FFT je tehdy maximálně efektivní. Velikost části filtru je zvolena $L = B$, pro další optimalizaci.

Každý blok vstupního signálu je transformovaný DFT pouze jednou, jeho spektrum je uloženo a znovupoužito na všechny části filtru. Jejich mezivýsledky jsou ve frekvenční doméně sečteny a inverzní DFT získáme výsledný blok výstupního signálu. Efektivita spočívá v minimalizaci počtu nutných transformací.

Zpoždění částí filtrů je možné realizovat ve frekvenční doméně, tato technika se nazývá zpožďovací linka ve frekvenční doméně (frequency-domain delay-line – FDL). Při převzetí nového vstupního bloku se již existující data v FDL posunou o jedno místo, a na první je uloženo spektrum vstupního signálu. Z toho vyplývá, že je možné v FDL zpožďovat data pouze o násobky B , z tohoto důvodu byla zvolena velikost částí filtru $L = B$.

Jednotlivé kroky algoritmu ilustrované na obrázku 1.6 jsou:

1. Filtr o velikosti N je rozdělen na $P = \lceil N/B \rceil$ částečných filtrů o délce B .
2. Každá část filtru je doplněna nulami na délku $2B$ a transformována FFT o velikosti $2B$. Výsledkem je symetrické spektrum $B + 1$ koeficientů.
3. Na vstupu FDL je vyrovnávací paměť o velikosti $2B$ vzorků, jejíž pravá polovina se přesune do levé a aktuální blok vstupního signálu se uloží do pravé.
4. Data v FDL se posunou o jedno místo, vstupní vyrovnávací paměť je transformována FFT o velikosti $2B$ a výsledné spektrum je uloženo do prvního místa FDL.
5. Jednotlivé páry spekter částí filtrů a dat v FDL se vynásobí a sečtou.
6. Konečně je vypočtena IFFT o velikosti $2B$, z výsledku velikosti $2B$ je levá polovina zahozena a na výstup jde blok o velikosti B .



Obr. 1.6: Schéma standardního algoritmu se segmentací o konstantní velikosti. [19]

2 Výsledky studentské práce

2.1 Metodika měření

Impulsní odezvy byly měřeny na dvou kytarových kombech a jednom kabinetu, třemi mikrofony, jmenovitě Shure SM57, Shure Beta 57A a měřícím mikrofonom. Měřeno bylo na 30 různých pozicích pro každý mikrofón, respektive 10 pozic v ose x s krokem 1 cm a 3 pozice v ose y : 0 cm, 10 cm, 40 cm. Počátek této soustavy byl zvolen na středu reproduktoru, ve vzdálenosti přibližně 5 cm od prachové vložky. Osa x vede horizontálně a je kolmá na osu reproduktoru, osa y je s osou reproduktoru shodná.

Pro měření impulsní odezvy byl využit analyzátor APx525, měřeno bylo sinusovým signálem s postupně zvyšující se frekvencí, výsledek zprůměrován ze tří měření.

Měření bylo provedeno v bezodrazové komoře FEKT VUT v Brně. Pro konzistentní posun v ose x byl využit XY držák na mikrofón se stupnicí.



Obr. 2.1: Měřicí ústrojí.

2.2 Výsledky měření

Naměřené impulsní odezvy byly zpracovány v MATLABu pro pozdější použití v aplikaci. Frekvenční odezvy získané Fourierovou transformací impulsních odezev odpovídají očekávaným hodnotám. Jelikož bylo měřeno i kalibrovaným měřícím mikrofónem, bylo by možné získat odezvy ostatních použitých mikrofónů, s mírnou odchylkou způsobenou nepřesnostmi v nastavení pozic mikrofónů.

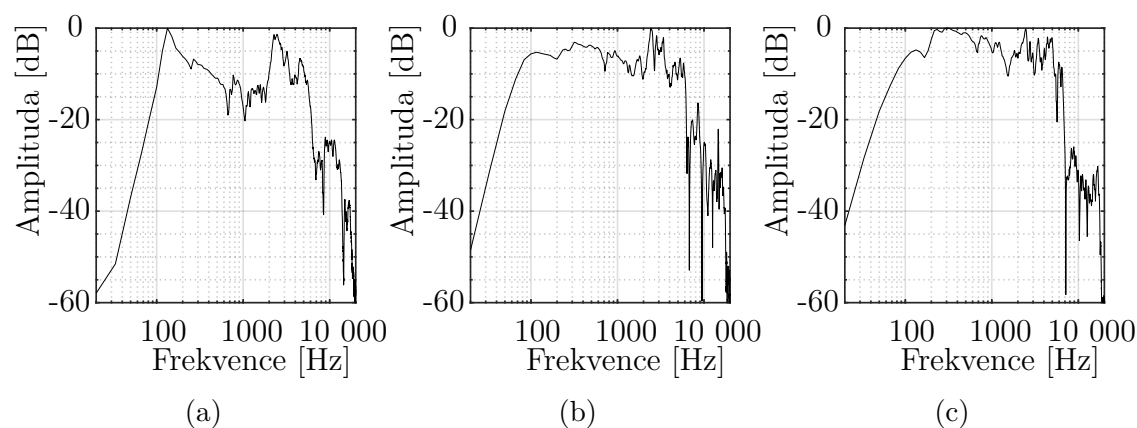
Na obr. 2.2 jsou zobrazeny charakteristiky měřených reproduktorů, s konstantní pozicí mikrofону. Značně se odlišuje reproduktor z Marshall kabinetu, má výraznou špičku na 130 Hz, v pásmu nižších středů je nevyrovnaný, další špička na 2,5 kHz odkud charakteristika rapidně klesá, v pásmu 6–20 kHz je silně zvlněná.

Oproti tomu Marshall kombo typu JCM200 DSL401 má vyrovnanější průběh do přibližně 3,5 kHz, špička na 2,5 kHz je přítomna, avšak ne tak výrazně jako u kabinetu. V pásmu 6–20 kHz je charakteristika opět zvlněná.

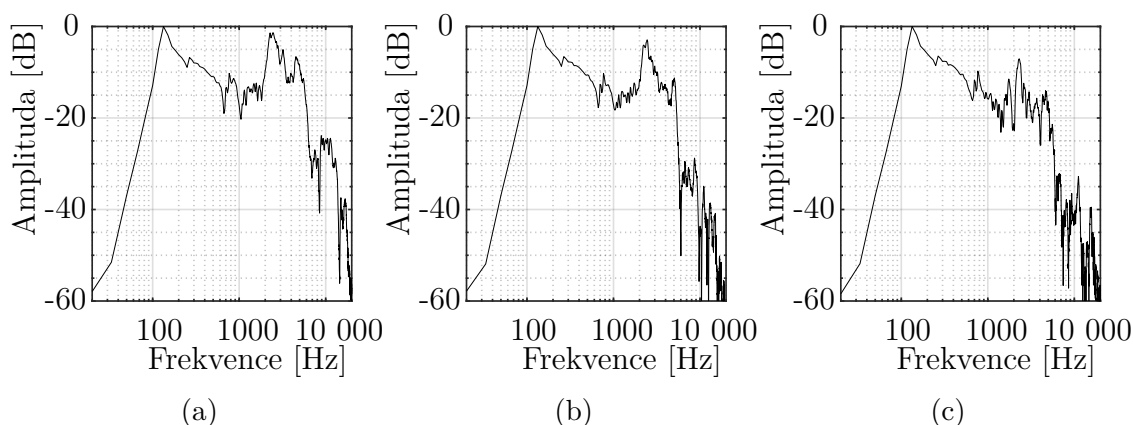
Kombo Line6 Spider Valve 112 vykazuje nejvyšší přenos již v pásmu 300–500 Hz, špička na 2,5 kHz je nejslabší ze všech tří reproduktorů. Po 6 kHz rapidně klesá a zvlněně stoupá do špičky na 16,5 kHz.

Co se týče změn způsobených pohybem v ose x , na obr. 2.3 je charakteristika kabinetu Marshall ve třech pozicích, s konstantní vzdáleností od reproduktoru. Posun od středu ke kraji celkově snižuje přenos vyšších frekvencí, toto snížení je pozorovatelné od 700 Hz.

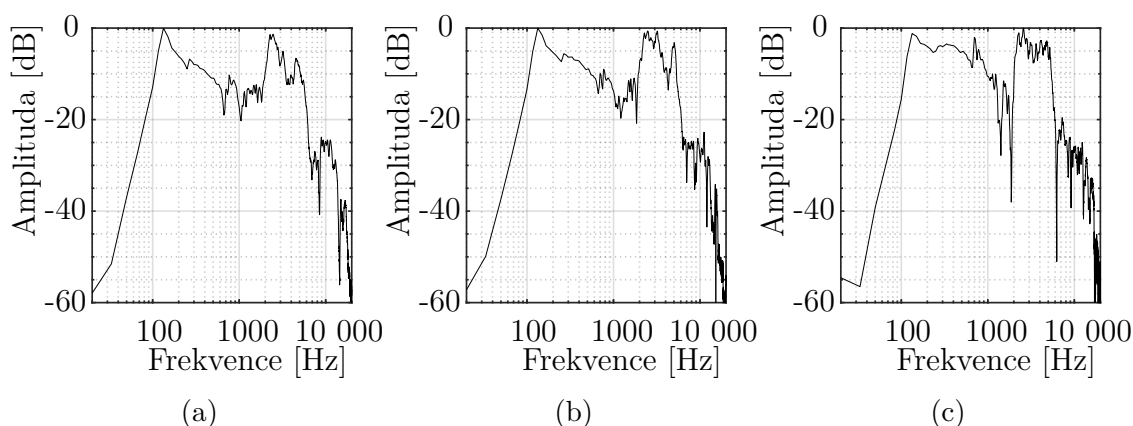
Porovnání změn způsobených polohou v ose y je na obr. 2.4, vyobrazené charakteristiky jsou pro kabinet Marshall s pozicí $x = 0$ cm. Poloha mikrofону v ose y má největší vliv na pásmo středních frekvencí, přesněji 130–5000 Hz, kde s rostoucí vzdáleností roste i přenos. Při vzdálenosti 10 cm se objeví špička na 5 kHz, ve vzdálenosti 40 cm je méně výrazná, ale stále přítomna. Frekvence nad 5 kHz jsou stále stejně zvlněny.



Obr. 2.2: Porovnání frekvenčních charakteristik měřených reproduktorů: (a) Marshall 4x12 kabinet, (b) Marshall JCM200 - DSL401, (c) Line6 Spider Valve 112. Pozice mikrofону konstantní ($x = 0$ cm, $y = 0$ cm).



Obr. 2.3: Frekvenční charakteristiky v pozicích: (a) $x = 0$ cm, (b) $x = 5$ cm, (c) $x = 10$ cm. Marshall kabinet, vzdálenost mikrofону konstantní ($y = 0$ cm).

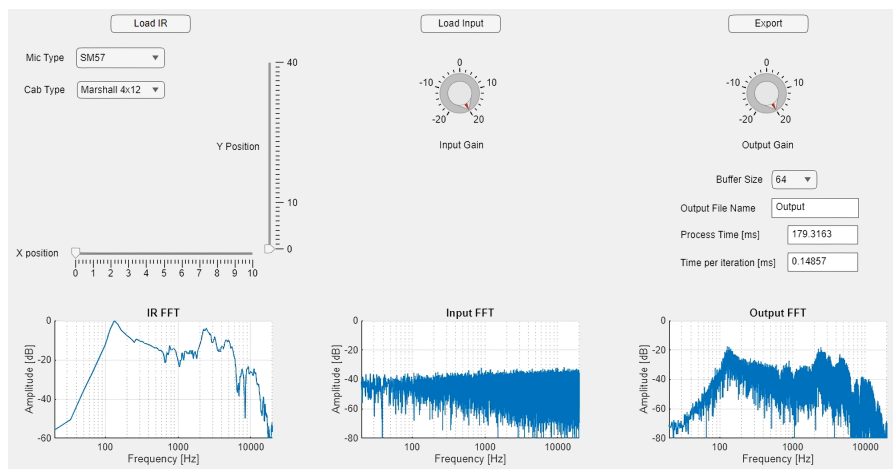


Obr. 2.4: Frekvenční charakteristiky v pozicích: (a) $y = 0$ cm, (b) $y = 10$ cm, (c) $y = 40$ cm. Marshall kabinet, pozice v ose x konstantní ($x = 0$ cm).

2.3 Implementace pomocí MATLAB App Designer

Simulátor byl vytvořen jako MATLAB aplikace pomocí modulu App Designer. Aplikace nepracuje v reálném čase, pouze ho vnitřní struktura simuluje. Aplikace umožňuje nahrát vlastní impulsní odezvu, či vybrat předem nahranou odezvu pomocí páru táhel a rozbalovacích menu, specifikujících typ komba, typ mikrofону a jeho polohu na ose x a y. Po nastavení je zobrazena kmitočtová charakteristika vypočtená FFT z impulsní odezvy. Podobně je zobrazeno spektrum uživatelem nahraného vstupního signálu a po kliknutí na tlačítko „Export“ i spektrum výstupního signálu. Rotační enkodéry slouží pro nastavení hodnoty, na kterou je vstupní, resp. výstupní signál normalizován. Posledně rozbalovací menu „Buffer Size“ nastavuje velikost

simulovaného bloku vzorků. Navržené uživatelské rozhraní je omezené možnostmi App Designeru, viz obr.2.5.



Obr. 2.5: Uživatelské rozhraní MATLAB aplikace.

2.3.1 Rozbor jednotlivých funkcí

Impulsní odezva

Při spuštění aplikace jsou pro rychlejší manipulaci načteny všechny naměřené impulsní odezvy ze souborů do vnitřní struktury. Následně se načte výchozí odezva a je zobrazena její frekvenční charakteristika.

Po změně hodnoty táhel či typu mikrofону nebo komba proběhne kontrola, zda je nutné odezvu interpolovat. V případě, že jsou táhla nastavena na hodnoty, ve kterých bylo provedeno měření, je adekvátní odezva načtena z vnitřní struktury. Jinak proběhne 1D interpolace mezi nejbližšími naměřenými odezvami pomocí MATLAB funkce `interp1`[25]. Poté je vykreslen graf nové frekvenční charakteristiky.

Tlačítko pro nahrání vlastní impulsní odezvy otevře okno pro výběr souboru. Soubory jsou omezeny na typ `.wav` nebo `.mat`. V případě wave souboru jsou načtena jeho data v mono konfiguraci a vzorkovací frekvence, data jsou normalizována na hodnotu 1 podle rovnice:

$$x_{IN} \cdot (A / \max(x_{IN})), \quad (2.1)$$

kde x_{IN} je vstupní signál a A je hodnota na kterou je normalizován. V případě souboru `.mat` je nutné z jeho vnitřní struktury získat potřebná data – v souboru jsou uloženy zvlášť časová data a amplitudy jednotlivých vzorků. Z časového posunu mezi deseti vzorky je vypočítána vzorkovací frekvence. Z množiny vzorků jsou poté odstraněny vzorky s negativním časem, zkrátí se na počet vzorků specifikovaný

délkou v milisekundách a nakonec jsou data normalizována. Z normalizovaných dat se pomocí FFT vykreslí frekvenční odezva.

Vstup

Obdobně pracuje funkce pro nahrání vstupního signálu, který je ovšem omezen pouze na soubory `.wav` a hodnota, na kterou je normalizován, je nastavitelná rotačním enkodérem v uživatelském prostředí. Poté je vykresleno frekvenční spektrum signálu.

Konvoluce

Konvoluce signálů je spouštěna tlačítkem „Export“. Nejprve je impulsní odezva převzorkována na vzorkovací frekvenci shodnou se vstupním signálem, následně je konvoluce realizována algoritmem popsáním v kapitole 1.3.1 ve smyčce simulující zpracování v reálném čase.

Vytvoří se nutný počet částečných filtrů podle délky impulsní odezvy a nastavené velikosti bloku vzorků, částečné filtry jsou nejprve inicializovány na potřebnou velikost nulami, poté naplněny daty ve smyčce. Následně jsou transformovány pomocí FFT. Inicializuje se zpožďovací linka FDL a matice, ve které probíhá násobení částečných filtrů se vstupním signálem. Poté je spuštěna smyčka, která simuluje předávání bloků vzorků vstupního signálu s jejich velikostí nastavitelnou v menu „Buffer size“. Ve smyčce je vytvořena pomocná paměť o dvojnásobné velikosti, se vstupními daty pouze v jedné polovině, zbytek je vyplněn nulami. FDL se posune o jedno místo, nový vstupní blok je transformován FFT a zapsán na začátek FDL. Poté je každý částečný filtr vynásoben s daty v odpovídající části FDL, výsledky násobení jsou sečteny, převedeny zpět do časové domény, a zapsány do výstupního souboru. Společně se spektrem výstupního signálu je zobrazen celkový čas, za který konvoluce proběhla, i průměrná doba výpočtu jednoho bloku vzorků.

2.4 Implementace v MATLAB knihovně Audio Toolbox

Knihovna Audio Toolbox mimo jiné obsahuje funkce a třídy pro realizaci modulů pracujících v reálném čase. Zde byla využita třída `audioPlugin` [26], která udává základní strukturu modulu, uživatel tak doplňuje pouze samotné algoritmy zpracování signálu a parametry modulu.

2.4.1 Zkreslení

Oproti aplikaci byl modul rozšířen o základní algoritmus zkreslení, využívající sigmoidní funkci:

$$f(x) = \frac{2}{1 + \exp(-ax) \cdot b} - 1, \quad (2.2)$$

kde x je vzorek vstupního signálu, a je uživatelem nastavitelný činitel zkreslení a b nastavitelná proměnná emulující přičtení DC (stejnoseměrné) složky ke vstupnímu signálu. Proměnná b zároveň určuje střidu výstupního signálu, který je obdélníkového charakteru. Vyššího zkreslení je možné dosáhnout zvýšením vstupní úrovně – hodnoty signálu, se kterými modul vnitřně pracuje, nejsou nijak omezeny.

Zkreslení signálu značně zvýší jeho úroveň, hodnota každého vzorku je tedy podělena aktuální hodnotou činitele zkreslení. Toto zaopatření proti přebuzení na výstupu není dokonalé, ale dostačující, jelikož má uživatel k dispozici rotační enkodér pro finální zesílení výstupního signálu.

Po zkreslení dojde ke konvoluci s vybranou impulsní odezvou. Algoritmus konvoluce byl již odzkoušen v aplikaci, zde je použit pouze s minimálními změnami týkající se zpracování chyb. Na rozdíl od aplikace modul neumožňuje nahrání vlastní impulsní odezvy.

2.4.2 Kmitočtový korektor

Další změnou oproti aplikaci je přidání emulace kmitočtového korektoru. Jelikož frekvenční odezva kmitočtového korektoru při neutrálním nastavení není lineární, jsou využity dva statické filtry typu pásmová propust a horní propust v paralelním zapojení, které tuto nelinearitu emulují. Mezní kmitočty pásmové propusti byly zvoleny 25, respektive 300 Hz, mezní kmitočet horní propusti 1300 Hz. Tyto hodnoty byly zvoleny na základě měření analogového zesilovače podle Texas Instruments [17] a Pirklovy implementace [2]. Objekty filtrů byly vytvořeny pomocí MATLAB funkce `fdesign`.

Po těchto filtrech jsou sériově zapojeny filtry ovládané enkodéry „Bass, Mid, Treble“. Jedná se o filtry low-shelving, peak a high-shelving, s mezními kmitočty přibližně převzatými z Pirklovy implementace [2], jmenovitě 180, 500, a 1250 Hz. Enkodéry ovládají jejich zesílení. Low shelf a high shelf filtry byly vytvořeny pomocí MATLAB funkce `shelvingFilter`, koeficienty peak filtru byly vypočteny podle 1.1. Následně pomocí funkce `dsp.IIRFilter` byl z koeficientů vytvořen objekt filtru.

Poslední změnou oproti aplikaci je přidání tlačítka pro vypnutí modulu, při jeho aktivaci je vstupní signál ovlivněn pouze vstupním a výstupním zesílením.

2.5 Implementace v C++ pomocí JUCE

Framework JUCE obsahuje třídy a funkce usnadňující vývoj modulů formátu VST3, AAX aj. Opět určuje hlavní strukturu modulu, za vývojáře obstarává mimo jiné předávání dat mezi hostitelskou aplikací a modulem, vývojář tedy řeší pouze zpracování příchozích dat a uživatelské prostředí.

Základní rozdělení pluginu JUCE je na třídu `PluginProcessor`, která operuje na audio vlákne, je zde tedy požadavek na rychlost kódu. Pokud není zpracování vstupního signálu provedeno za dobu udanou vzorkovací frekvencí a velikostí bloku předávaných vzorků, není zaručen obsah výstupního signálu – může dojít k výpadku či blok vzorků může obsahovat nesmyslná data. Oproti tomu třída `PluginEditor` zajišťuje uživatelské rozhraní, nejsou zde nároky na rychlost kódu a pracuje na jiném vlákne.

Z důvodu, že tyto dvě třídy nepracují na stejném vlákne, je vhodné omezit předávání dat mezi nimi na minimum a předávání mít zabezpečené. Z tohoto důvodu jsou využívány atomické proměnné, které nelze změnit ve více vláknech najednou.

2.5.1 Zpracování signálu

Pro zpracování vstupního signálu slouží v modulu třída `PluginProcessor`, sestávající z vícero funkcí. Důležité z nich budou dále vysvětleny v pořadí, v jakém jsou volány při spuštění modulu či odpovídající signálové cestě. Dále jsou v této třídě zapouzdřeny objekty vlastních tříd, které zajišťují zpracování signálu, tyto objekty jsou v textu nazývány procesory.

Při vytvoření instance modulu je nejprve volána funkce `setStateInformation`, která se pokusí nahrát uložené nastavení modulu v případě že modul není nově přidán (např. při otevření uloženého projektu hostitelského softwaru). Pokud jsou uložená nastavení nahrána, proběhne zde aktualizace procesorů filtrů, nahrání přibalených impulsních odezev do paměti a do procesoru konvoluce.

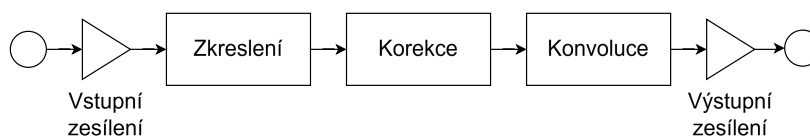
Funkce `loadShippedImpulseResponses` zajišťuje nahrávání impulsních odezev do paměti. Nejprve připraví prázdnou 4D matici, do které jsou následně ukládány jednotlivé soubory odezev. Soubory jsou nahrány do paměti pro rychlejší přístup. Modul očekává odezvy ve složce `C:\ProgramData`, což je složka standardně využívána moduly VST3 pro tyto účely. Dochází zde také ke kontrole, zda je složka souborů kompletní. Výsledek kontroly je uložen do atomické proměnné, ke které má přístup i `PluginEditor`.

Nahrání impulsní odezvy do objektu konvolučního procesoru probíhá ve funkci `updateLoadedIR`, jejíž provedení může trvat příliš dlouho, je tedy pro zamezení chyb ve výstupním signálu vypnuté jeho zpracování. Nejprve proběhne kontrola,

zda je nutné odezvu interpolovat. Pokud je uživatelem zvolena pozice mikrofonů odpovídající nějakému z naměřených souborů, je do konvolučního procesoru nahrán odpovídající soubor z matice v paměti. Pokud je nutné interpolovat, proběhne 2D interpolace mezi čtyřmi nejbližšími soubory odezev a výsledný soubor je nahrán do konvolučního procesoru.

Dále proběhne funkce `prepareToPlay`, ve které dochází k inicializaci všech procesorů. Předem se zde alokuje paměť pro pomocné proměnné, nastaví se vnitřní proměnné procesorů, předává se jim vzorkovací frekvence a velikost vstupního bloku vzorků a nahrají se přibalené impulsní odezvy. Touto funkcí je dokončena inicializace modulu ze strany zpracování signálu.

Závěrečné zpracování bloku vstupních vzorků probíhá ve funkci `processBlock`. Signálová cesta je zobrazena na obr. 2.6. Po vstupním zesílení je detekována maximální hodnota vzorku v obou kanálech, která je využita pro indikaci v uživatelském prostředí. Zkreslení, korekce a konvoluce proběhnou na základě nastavení ovládacích prvků. Následně je na blok aplikována konvoluce a výstupní zesílení. Z výsledného bloku je zjištěna maximální hodnota obou kanálů a zároveň je blok uložen do FIFO (first in first out) kontejneru pro FFT analýzu.



Obr. 2.6: Blokové schéma modulu.

2.5.2 Vstupní a výstupní zesílení

Blok vstupního a výstupního zesílení je realizován pomocí třídy `JUCE Gain` [27]. Třída zapouzdřuje funkce pro nastavení zesílení, i pro přepočítání mezi činitelem zesílení a decibely a funkci, která toto zesílení aplikuje na vzorky, či na celý blok vzorků. Dále disponuje možností uhlazení změn hodnoty zesílení s nastavitelnou rychlostí. Toto předchází artefaktům, které mohou ve výstupním signálu vzniknout při rychlé manipulaci s napojeným prvkem v uživatelském prostředí.

2.5.3 Zkreslení

Modul obsahuje tři algoritmy zkreslení, každý je realizován ve vlastní třídě, které jsou zabaleny do třídy `Distortion`, kde probíhá převzorkování a úprava úrovně. Každý algoritmus způsobí jiné zabarvení signálu, které je volně inspirováno nastaveními na klasickém kytarovém zesilovači (Clean, Crunch, Lead).

Pro dosažení většího zkreslení je signál před zkreslením zesílen, míra je závislá na ovládacím prvku zkreslení „Drive“. Po zkreslení je naopak signál utlumen, aby nepřesáhnul mimo interval $\langle -1, 1 \rangle$. Zesílení je opět provedeno objekty třídy `Gain`.

Zde je nutné odbočit k vysvětlení frekvenčního spektra vzorkovaného signálu. Spektrum periodického signálu je symetrické podle nulové frekvence. Jakmile je signál navzorkovaný, spektrum je navíc periodické s periodou rovnou vzorkovací frekvenci [18]. Z toho vyplývá, že pokud signál obsahuje frekvence vyšší než je polovina vzorkovací frekvence, dojde k překrytí spekter – aliasingu.

Harmonické složky vytvořené zkreslením signálu mohou způsobit aliasing. Z toho důvodu je zavedeno 4násobné nadvzorkování vstupního signálu, a jeho decimace po zkreslení. Při decimaci je signál filtrován dolní propustí, která odstraní frekvenční složky, které by jinak způsobily aliasing. Převzorkování je provedeno JUCE třídou `Oversampling`.

Yamaha třída B

Tento algoritmus byl převzat z patentu firmy Yamaha [14] a implementace navržené Pirklem [2]. Patent popisuje emulaci zesilovače třídy B, kdy je signál rozdělen do dvou větví, které jsou téměř identické, v jedné z nich je navíc fázový invertor. K signálu je přičtena stejnosměrná složka, emulující posun pracovního bodu triody a následně je zpracován waveshaperem, emulujícím přenosovou funkci triody.

Vlastní implementace nepřičítá k signálu DC složku, ale využívá sigmoidní funkci 2.2, kde proměnná b emuluje přičtení DC složky. Tato proměnná je pohyblivá a její hodnota závisí na velikosti zkreslení nastavené uživatelem, stejně jako proměnná a , která udává strmost funkce a tím obdélníkový charakter výstupní vlny.

Polettiho třída B

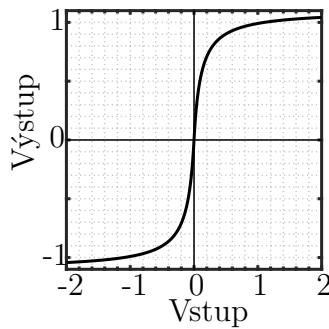
Algoritmus byl navržený podle Polettiho žádosti o patent [15] a Pirklovy upravené implementace [2]. Originální algoritmus opět signál rozdělí do dvou větví. Ve větvi je nejprve waveshaper s asymetrickou přenosovou funkcí, následně je odfiltrována DC složka a nakonec je signál zpracovaný waveshaperem se symetrickou přenosovou funkcí. Všechny waveshapery využívají stejnou funkci, mění se pouze hodnoty

proměnných. Předpis funkce je:

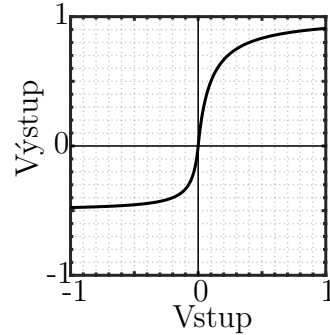
$$y(n) = \begin{cases} \frac{kx(n)}{1 - \frac{kx(n)}{L_n}} & x(n) \leq 0, \\ \frac{kx(n)}{1 + \frac{kx(n)}{L_p}} & x(n) > 0 \end{cases} \quad (2.3)$$

Funkce obsahuje parametr k , který udává strmost funkce a parametry L_n, L_p udávající dolní, respektive vrchní limity. Pokud je $L_n = L_p$, funkce je symetrická (lichá). Ve vlastní implementaci jsou hodnoty limit nastaveny na 1,1 pro symetrický waveshaper, asymetrické waveshapery mají nastaveny hodnoty na 23,6 a 0,5 [2] tak, že jsou mezi nimi hodnoty L_n a L_p prohozeny.

Parametr k je vypočtený z hodnoty rotačního enkodéru „Drive“, jeho výpočet se liší mezi symetrickým a asymetrickým waveshaperem, ale maximální hodnota je u obou rovna 40. U symetrického waveshaperu stoupá k exponenciálně, u asymetrického lineárně. DC složka je odstraněna filtrem typu horní propust s mezním kmitočtem 5 Hz.



(a) $k = 10, L_p = L_n = 1.1$



(b) $k = 10, L_p = 1, L_n = 0.5$

Obr. 2.7: Přenosové funkce Polettiho algoritmu.

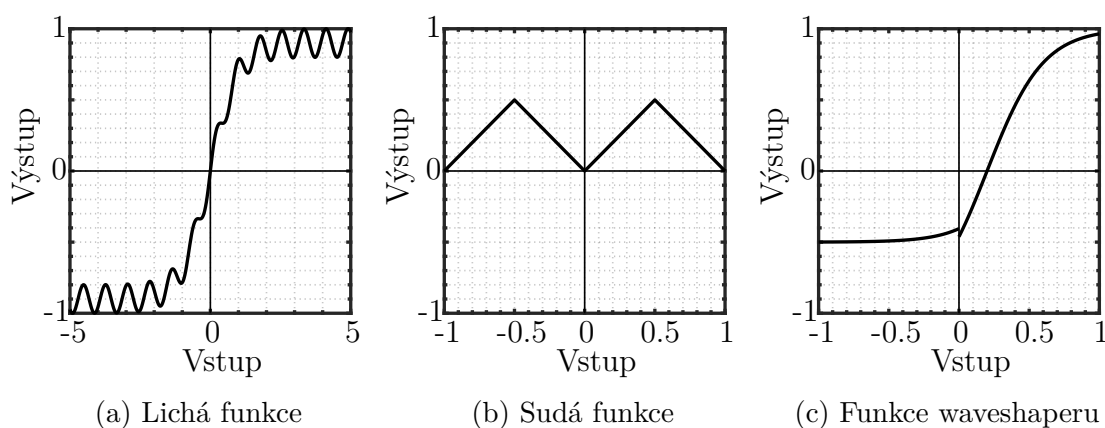
Wavefolder

Třetí algoritmus zkreslení je čistě experimentální, využívá podobné struktury jako Polettiho algoritmus. Signál je zpracován dvěma paralelními wavefoldery, jejich výstup je sečten a zpracován waveshaperem. Wavefolder je speciální případ waveshaperu, kde dochází k překlopení vstupní vlny – přenosová funkce není monotónní. Nakonec je ze signálu vyfiltrována stejnosměrná složka, opět filtrem typu horní propust s mezním kmitočtem 5 Hz.

Každý wavefolder má vlastní přenosovou funkci, je využita lichá a sudá funkce pro dosažení plnějšího spektra. Sudá funkce je trojúhelníkového charakteru s předpisem $y(n) = |x(n) - \lfloor x(n) \rfloor|$, kde $\lfloor x(n) \rfloor$ je zaokrouhlení hodnoty vzorku na nejbližší celočíselnou hodnotu. Předpis liché funkce je $0,9 \cdot \tanh(x(n)) + 0,1 \cdot \sin(8x(n))$.

Waveshaper využívá obecnou funkci pro generování plnějšího spektra, která navíc není kontinuální – v bodě, kde je $x(n)$ rovnou nule je skok mezi kladnou a zápornou polovinou funkce, viz obr. 2.8c. Předpis funkce je:

$$y(n) = \begin{cases} \frac{2}{1 + \exp(-5x(n) + 1)} - 1 & x(n) > 0, \\ \frac{2}{1 + \exp(-5x(n) + 3)} - 0,5 & x(n) \leq 0 \end{cases} \quad (2.4)$$



Obr. 2.8: Přenosové funkce využívané ve Wavefolder typu zkreslení.

2.5.4 Korekční zesilovač

Korekční zesilovač je realizován stejným způsobem jako v kap. 2.4.2. Koeficienty všech pěti použitých filtrů jsou vypočteny pomocí funkcí obsažených v JUCE třídě `IIR::Coefficients` [28]. Jednotlivé objekty filtrů jsou zabaleny ve vlastní třídě `ToneStack`, která zprostředkovává zpracování signálu filtry ve správném pořadí. Její objekt je obsažen ve třídě `PluginProcessor`, odkud jsou volány funkce na přípravu a předávání se zde blok vstupních vzorků.

2.5.5 Konvoluce

Pro konvoluci s impulsní odezvou byl využit objekt třídy JUCE `Convolution` [22]. V něm jsou zapouzdřeny mimo jiné funkce pro načtení impulsní odezvy s podporou

různých datových typů a funkce provádějící konvoluci, které využívají algoritmu Overlap-Add se segmentací o konstantní velikosti a nulovým zpožděním.

Při zpracování každého vstupního bloku dochází k několika kontrolám: zda by měl být procesor konvoluce vypnutý, podle stavu proměnné navázané na uživatelské prostředí, zda je v procesoru nahrána impulsní odezva o nenulové délce a zda je nutné odezvu změnit. Způsob změny impulsní odezvy již byl diskutován v kap. 2.5.1. Skutečnost, jestli je změna nutná, je udávána atomickou proměnnou napojenou na prvky v uživatelském prostředí.

2.6 Uživatelské rozhraní

Stejně jako u zpracování signálu je zde jedna hlavní třída `PluginEditor`, která zapouzdřuje objekty jednotlivých komponent uživatelského prostředí, jako jsou rotační enkodéry, tlačítka aj. Dále obsahuje funkce `paint`, kde probíhá vykreslení pozadí a `resized` kde jsou komponenty rozloženy.

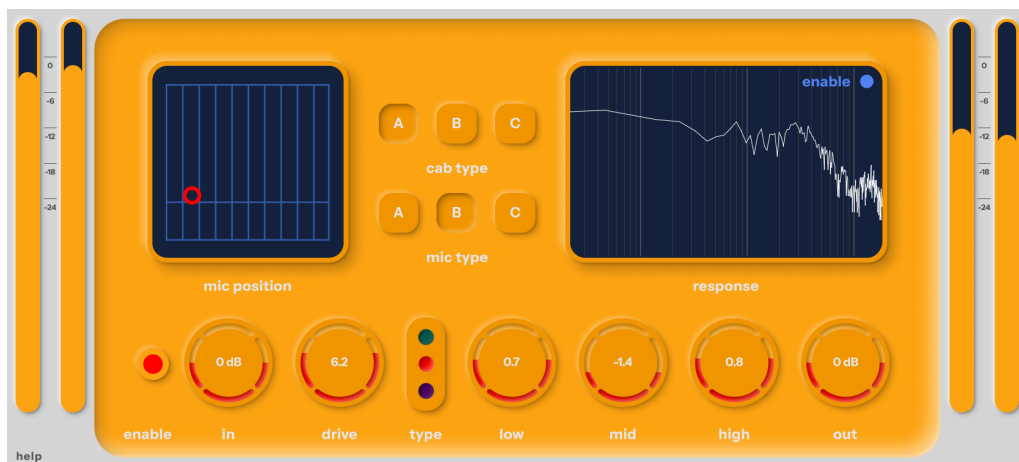
V konstruktoru třídy probíhá inicializace komponent a jejich napojení na proměnné třídy `PluginProcessor`. Některé komponenty nejsou přímo napojeny na proměnné, ale je využita lambda – anonymní funkce definovaná přímo v místě, kde je předána objektu – která je volána např. při změně hodnoty (`onValueChange`) nebo při kliknutí na tlačítko (`onClick`). Dále je v konstruktoru nastavena velikost okna, možnost měnit jeho velikost a fixní poměr stran okna. Posledně se v případě nekompletních přibalených dat vykreslí chybové hlášení a spouští se časovače, s jejichž frekvencí se aktualizují komponenty.

Rozložení komponentů není dáno fixní pozicí komponentu, ale je relativní vůči velikosti okna, aby bylo možné měnit jeho velikost. Toho je docíleno postupným odstraňováním částí okna, kdy velikost odstraněné části je udávána procenty z velikosti okna, či jeho části. Komponentům je možné přiřadit odstraněnou či zbylou část okna, jako hranice, kde mají být vykresleny.

Komponenty využívají tříd dostupných v knihovně JUCE, například tlačítka `ToggleButton` a rolovací menu `ComboBox` nebylo nutné modifikovat. Jiné komponenty, kde bylo nutné modifikovat jejich funkce či přidat proměnné, dědí z obecné třídy `Component` [29]. Důležitější z nich budou dále rozebrány do větší hloubky.

Samotný vzhled komponentů je nastaven pomocí vlastních tříd dědicích z JUCE třídy `LookAndFeel` [30]. Ve vlastní třídě jsou vždy přepsány funkce zajišťující vykreslení komponent, které jsou závislé na typu komponentu. Pokud je tedy v modulu vícero komponent stejného typu, které mají mít rozdílný vzhled, je nutné mít více tříd `LookAndFeel`. Statické části komponent byly předem navrženy v grafickém softwaru a v modulu jsou vykresleny ze souborů PNG (Portable Network Graphics). Dynamické části, například výplň rotačních enkodérů, jsou vykreslovány vektorově

přímo v kódu. Využit byl volně dostupný font Instrument Sans [31]. Uživatelské prostředí je vyobrazeno na obr. 2.9.



Obr. 2.9: Grafické rozhraní modulu VST3.

Rotační enkodér

Knihovna JUCE obsahuje třídu `Slider`, která je určena pro všechny typy enkodérů. Komponent z této třídy dědí a rozšiřuje ji o funkce nastavující text zobrazující hodnotu enkodéru, případně odpovídající jednotku.

XY ovladač

Pozice mikrofону je nastavitelná ve dvou osách, z toho důvodu byl zvolen XY ovladač, který zároveň lépe reprezentuje nastavenou pozici oproti kombinaci dvou rotačních enkodérů. Kód komponentu byl založen na repozitáři [32]. Ovladač je nepřímo napojen na proměnné prostřednictvím enkodérů, které nejsou vykreslovány. Tento způsob umožňuje například ovládat více proměnných s rozdílnými rozsahy pohybem v jedné společné ose.

Ovladač sestává z třídy `XYPad`, která zapouzdřuje metody pro vykreslování komponent a metod pro přiřazení enkodérů k osám ovladače. Druhou součástí je třída `Thumb`, což je část komponentu, se kterou uživatel interaguje pro nastavení pozice mikrofону. Sestává opět z metod pro vykreslení a metod definujících chování při tažení myši.

Zobrazení impulsní odezvy

Pro lepší zpětnou vazbu je uživateli zobrazena frekvenční charakteristika aktuálně zvolené impulsní odezvy. Za tímto účelem byly vytvořeny tři třídy na základě kódu v repozitáři [33].

Impulsní odezva je převedena do frekvenční domény za pomoci FFT ve třídě `FFTDataGenerator`. Z transformovaných dat je poté ve třídě `PathProducer` vytvořena množina bodů, jejichž vykreslením ve třídě `IrFFTComponent` vznikne křivka zobrazená uživateli. Také je zde vykreslena frekvenční mřížka pro lepší orientaci v grafu.

Zobrazení úrovní signálu

Modul zobrazuje špičkovou úroveň vstupního a výstupního signálu, úroveň je detekována v každém zpracovaném bloku vzorků. Jelikož by bylo zobrazení příliš citlivé, je zavedeno uhlazení úrovní v případě, že se jedná o úroveň nižší, než je aktuálně zobrazená. Pro věrné zobrazení přechodných špičkových hodnot není hodnota úrovně uhlazena pokud je vyšší, než je úroveň aktuálně zobrazená. Nakonec je vykreslen zaoblený obdélník s výškou odpovídající úrovni přepočtené na decibely. V případě přebuzení je změněna barva vykresleného obdélníku. To vše je zapouzdřeno ve třídě `HorizontalMeter`.

2.6.1 Instalace

Jelikož modul vyžaduje pro správnou činnost přibalené impulsní odezvy ve specifické složce v systémovém adresáři, byl pro zjednodušení instalačního procesu vytvořen instalační skript za použití aplikace Inno Setup Compiler [34]. Skript po spuštění automaticky umístí soubory odezev do požadované složky, a zároveň soubor `.vst3` do adresáře dedikovaného pro VST3 moduly. Vytvořen je i skript pro automatickou odinstalaci modulu a smazání souvisejících souborů.

2.7 Testování VST3 modulu

Pro testování modulu bylo využito programů MathWorks MATLAB R2024b, Cockos REAPER a DDMF PluginDoctor. Byly naměřeny harmonické složky vzniklé zkreslením a frekvenční odezva filtrů korekčního zesilovače.

2.7.1 Analýza zkreslení

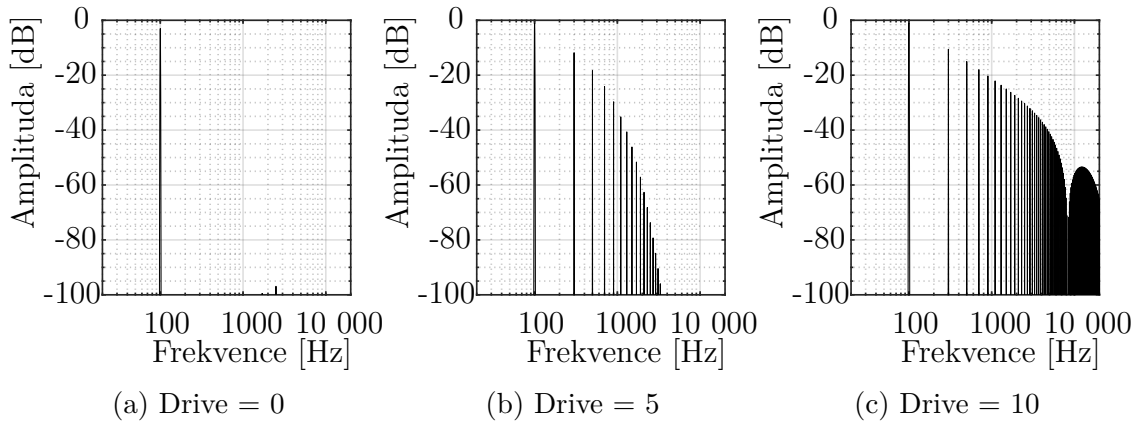
Jako vstupní signál byl vybrán harmonický signál s amplitudou 0 dB a frekvencí 100 Hz. Pro každý typ zkreslení bylo měření provedeno se třemi nastaveními enkóderu „Gain“ – minimum, střed, maximum. Jelikož změnou nastavení zesílení dojde i ke změně amplitudy výstupního signálu, byly naměřené vzorky normalizovány. Dále bylo vypočteno jejich THD (Total Harmonic Distortion). Pro výpočet THD byla využita MATLAB funkce `thd`, která pracuje na základě definice obvykle označované

THD_F [35]:

$$\text{THD}_F = \frac{\sqrt{\sum_{n=2}^{\infty} I_n^2}}{I_1}, \quad (2.5)$$

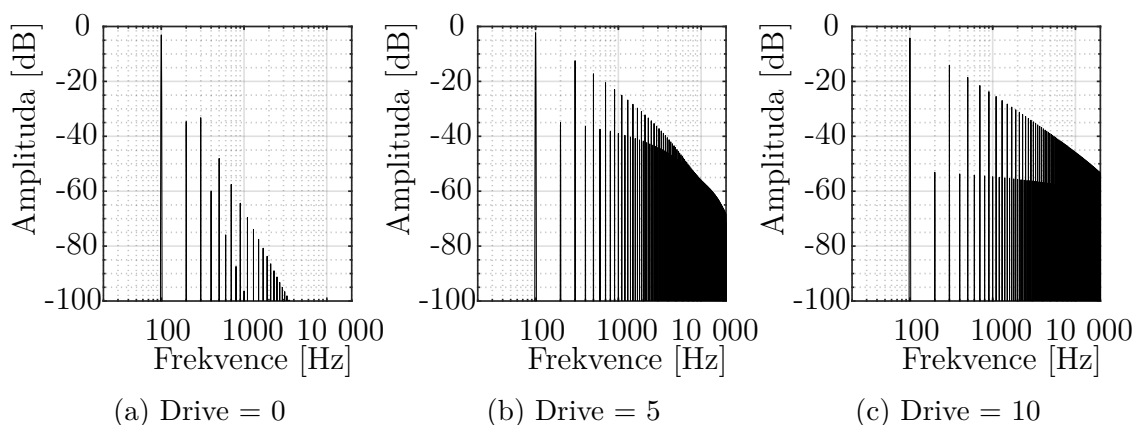
kde I_n je amplituda n -té harmonické složky. Vypočtené hodnoty THD jsou v tab. 2.1.

Algoritmus podle Yamahy generuje nejméně harmonických složek, při nízkých nastaveních enkodéru „Drive“ je přenos prakticky lineární, čemuž odpovídá i hodnota THD. Vznikají pouze liché harmonické, jejichž obálka při vyšším nastavení zkreslení připomíná funkci sinc. Frekvenční spektra výstupních signálů jsou vyobrazena na obr. 2.10.



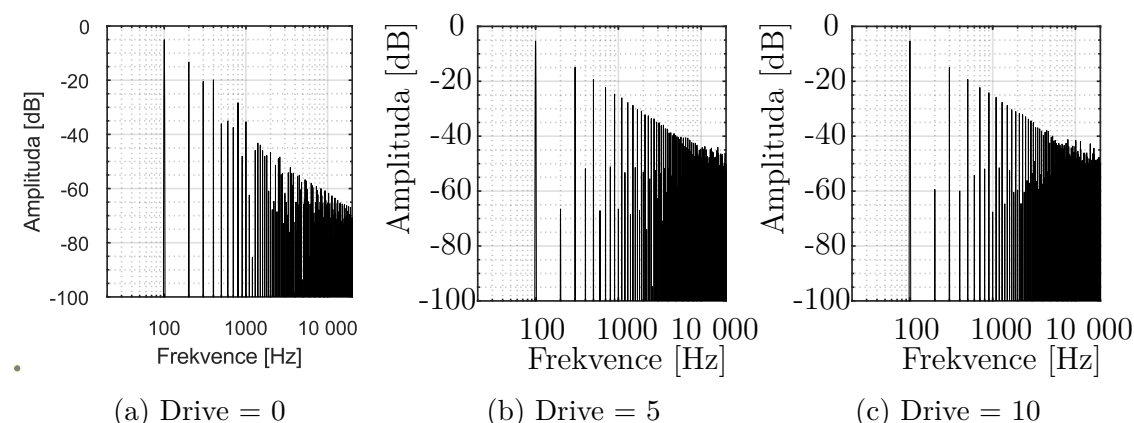
Obr. 2.10: Frekvenční spektra tvořená algoritmem podle Yamahy (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.

Polettiho algoritmus generuje díky použití asymetrických přenosových funkcí sudé i liché harmonické složky, viz obr. 2.11. Obálka lichých harmonických klesá pro všechna nastavení zkreslení prakticky lineárně, amplituda sudých harmonických složek s vyšším nastavením zkreslení klesá. Maximální naměřené THD není tak vysoké, jak u algoritmu Yamahy, avšak díky přítomnosti všech harmonických složek má signál plnější barvu.



Obr. 2.11: Spektra tvořená algoritmem podle Polettiho (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.

Zkreslení s využitím wavefolderů generuje nejvíce zabarvený signál, který má plné spektrum již při nejnižším nastavení zkreslení, viz obr. 2.12. Při vyšších zkresleních dochází k útlumu sudých harmonických složek. Tento mód zkreslení generuje velmi podobné THD pro všechna nastavení zkreslení, zvuková barva je však odlišná.



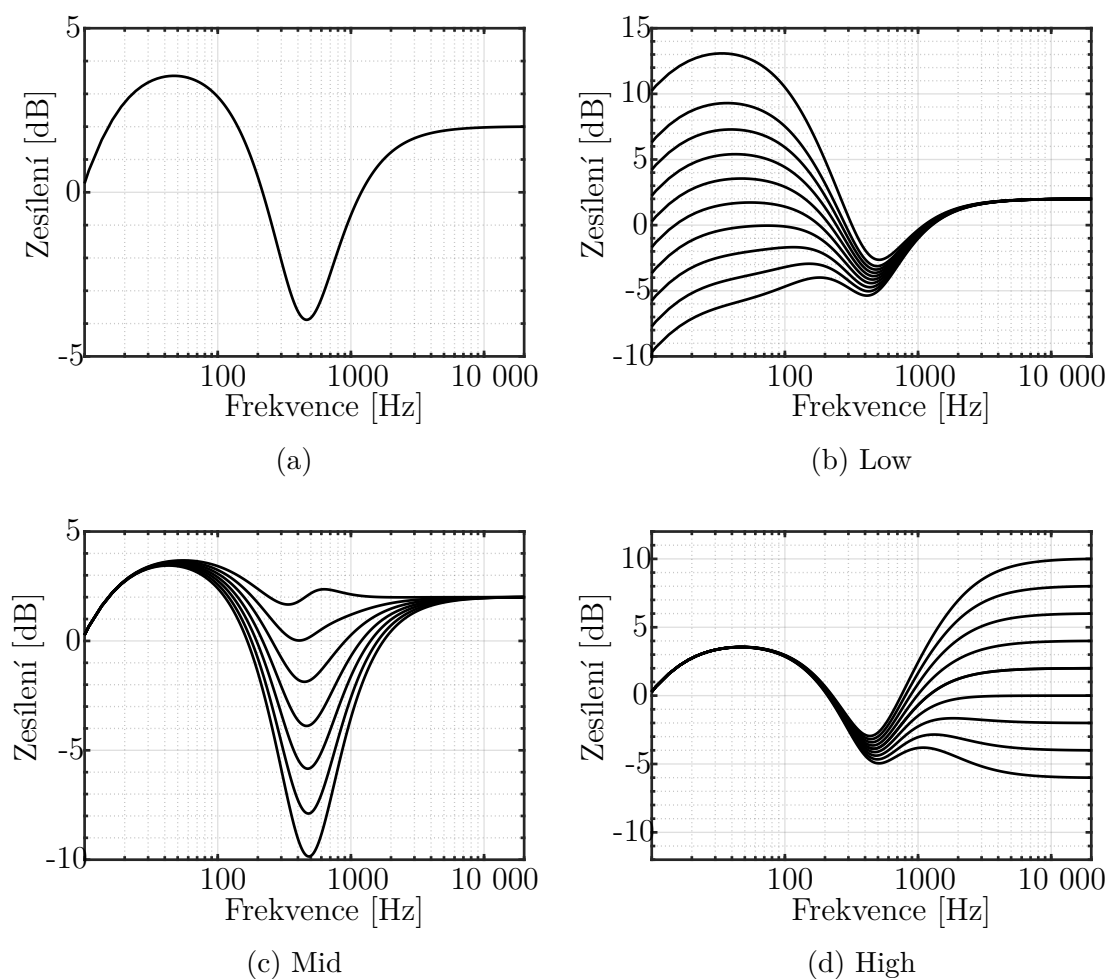
Obr. 2.12: Spektra tvořená algoritmem Wavefolder (kap. 2.5.3) pro různá nastavení enkodéru „Drive“.

THD _F [%]	Yamaha	Poletti	Waveshaper
Min	0,006	4,13	46,3
Střed	33,24	41,79	48,25
Max	46,3	45,91	48,29

Tab. 2.1: Hodnoty THD pro daný typ zkreslení a nastavení enkodéru „Drive“.

2.7.2 Odezva filtrů korektoru

Odezva filtrů byla naměřena pro plný rozsah ovládacích prvků, pro každý prvek zvlášť. Odezva korekčního zesilovače se všemi ovládacími prvky nastavenými na středu je na obr. 2.13a. Tato odezva přibližně odpovídá měření v [17]. Vliv jednotlivých ovládacích prvků na výsledný přenos je na obr. 2.13. Pro naměření odezvy filtrů byla využita lineární analýza modulu v programu PluginDoctor.



Obr. 2.13: Odezvy korekčního zesilovače v neutrálním stavu (a) a pro různá nastavení jednotlivých enkodérů (b–d).

Závěr

V práci byly shrnuty teoretické poznatky důležité pro návrh a realizaci výsledného modulu, zmíněna metodika měření a okomentovány výsledky měření.

Dále byly důkladně rozepsány všechny výsledné programy, jak z MATLAB prostředí, tak VST modul, vysvětleny vnitřní procesy jednotlivých funkcí, kde jsou v kódu volány a jaký je jejich účel.

V poslední kapitole byla provedena analýza VST modulu – byly porovnány frekvenční spektra jednotlivých módů zkreslení a naměřeny odezvy filtrů kmitočtového korektoru.

Literatura

- [1] DARR, Jack. *Electric Guitar Amplifier Handbook*. 3rd edition, 2nd printing. Sams, 1973. ISBN 0-672-20848-2.
- [2] PIRKLE, Will. *Chapter 19 Addendum: Vacuum Tube & Distortion Emulation*. Online. 2019 [cit. 2025-05-17]. Dostupné z:
<https://www.willpirkle.com/fx-book-bonus-material/chapter-19-addendum/>
- [3] A.F. Double Triode ECC83. 1970. Dostupné z:
<https://frank.pocnet.net/sheets/010/e/ECC83.pdf>. [cit. 2025-05-18].
- [4] KUBÁNEK, David. *Návrh a konstrukce zvukové techniky: Elektronkové zesilovače*. Online. Brno: Vysoké učení technické [cit. 2025-05-18]
- [5] GODBOLD, Nat Hartwell. *Hysteresis Effects In Tetrodes*. Online, Disertace. Austin, Texas: University of Texas, 1937. Dostupné z:
<https://hdl.handle.net/2152/126905>. [cit. 2025-05-18].
- [6] BALLANTINE, Stuart a COBB, H. L. Power Output Characteristics of the Pentode. Online. In: *Proceedings of the Institute of Radio Engineers*. 1930, s. 450-470. ISSN 2162-6626. Dostupné z:
10.1109/JRPROC.1930.222023. [cit. 2025-05-18].
- [7] KRATOCHVÍL, Tomáš. *Audio Elektronika: Předzesilovače*. Online. Brno: Vysoké učení technické [cit. 2025-05-11]
- [8] JUNGSMANN, Thomas. *Theoretical and practical studies on the behaviour of electric guitar pick-ups* [online]. Helsinki, 1994 [cit. 2025-05-17]. Dostupné z:
http://research.spa.aalto.fi/publications/theses/jungsmann_mst.pdf. Diplomová práce. Helsinki University of Technology.
- [9] KEEPORTS, David. The warm, rich sound of valve guitar amplifiers. *Physics Education* [online]. 2017, **52**(2), 1-7 [cit. 2025-05-17]. Dostupné z:
<https://dx.doi.org/10.1088/1361-6552/aa57b7>.
- [10] ROADS, Curtis. A Tutorial on Non-Linear Distortion or Waveshaping Synthesis. *Computer Music Journal*. 1979, roč. 3, č. 2, s. 29-34.
- [11] PAKARINEN, Jyri a YEH, David T. A Review of Digital Techniques for Modeling Vacuum-Tube Guitar Amplifiers. *Computer Music Journal*. 2009, roč. 33, č. 2, s. 85–100. JSTOR. Dostupné z:
<http://www.jstor.org/stable/40301029>. [cit. 2025-01-2]

- [12] ZÖLZER, Udo (ed.). *DAFX: digital audio effects*. 2nd ed. Chichester: Wiley, c2011. ISBN 978-0-470-66599-2.
- [13] SCHIMMEL, Jiří. *Studiová a hudební elektronika*. Online. Druhé. Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav komunikací, 2015. ISBN 978-80-214-4452-2. Dostupné z:
https://www.vut.cz/www_base/priloha_fs.php?dpid=61387&skupina=dokument_priloha. [cit. 2025-05-19].
- [14] YAMAHA CORPORATION (US). *DIGITAL AUDIO SIGNAL PROCESSOR WITH HARMONICS MODIFICATION*. Vynálezce Ryuichiro KUROKI, Tsugio ITO. Přihl.: 18.1.1996. Uděl.: 24.11.1998. 5841875.
- [15] *NONLINEAR PROCESSOR FOR AUDIO SIGNALS* (US). Vynálezce Mark Allstair POLETTI. Přihl.: 22.8.2007. Uděl.: 28.2.2008. US 2008/0049950 A1.
- [16] VANHATALO, Tara et al. *A Review of Neural Network-Based Emulation of Guitar Amplifiers*. Online. Appl. Sci. 2022, roč. 12, č. 12, s. 5894. Dostupné z:
<https://doi.org/10.3390/app12125894>. [cit. 2025-05-19].
- [17] WILLIAMS, Ian. *Tone Stack for Guitar Amplifier Reference Design*. Online. Texas Instruments. 2015. Dostupné z:
<https://www.ti.com/lit/ug/tidu887/tidu887.pdf>. [cit. 2025-05-23].
- [18] SMÉKAL, Zdeněk. *Analýza signálů a soustav - BASS*. Online. Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav komunikací, 2012. Dostupné z:
<https://moodle.vut.cz/mod/resource/view.php?id=214606>. [cit. 2025-05-19].
- [19] WEFERS, Frank. *Partitioned convolution algorithms for real-time auralization*. Online, Disertace. Berlin: Rheinisch-Westfälische Technische Hochschule Aachen, 2015. Dostupné z:
<https://publications.rwth-aachen.de/record/466561>. [cit. 2023-12-11].
- [20] SMÉKAL, Zdeněk. *Analýza signálů a soustav: Signály s diskrétním časem*. Online. Brno: Vysoké učení technické [cit. 2023-12-10]
- [21] STOCKHAM JR., Thomas Greenway. High-speed convolution and correlation In: *Proceedings of the April 26-28, 1966, Spring joint computer conference*. 1966, roč. 28. s. 229–233
- [22] *JUCE dokumentace třídy Convolution*. Online. Dostupné z:
https://docs.juce.com/master/classdsp_1_1Convolution.html. [cit. 2025-05-21].

- [23] KUNDUR, Deepa. *Overlap-Save and Overlap-Add*. Online. Toronto: University of Toronto. Dostupné z:
https://ww2.comm.utoronto.ca/~dkundur//course_info/real-time-DSP/notes/8_Kundur_Overlap_Save_Add.pdf. [cit. 2023-12-11].
- [24] SAYOLS, Narcís. *Mathematical methods of signal processing*. Online, Diplomová práce. Universitat Politècnica de Catalunya, 2011. Dostupné z:
<http://hdl.handle.net/2099.1/14560>. [cit. 2023-12-11].
- [25] *MATLAB dokumentace funkce interp1*. Online. Dostupné z:
<https://www.mathworks.com/help/matlab/ref/double.interp1.html>. [cit. 2025-05-21].
- [26] *MATLAB dokumentace funkce interp1*. Online. Dostupné z:
<https://www.mathworks.com/help/audio/ref/audioplugin-class.html>. [cit. 2025-05-21].
- [27] *JUCE dokumentace třídy Gain*. Online. Dostupné z:
https://docs.juce.com/master/classdsp_1_1Gain.html. [cit. 2025-05-21].
- [28] *JUCE dokumentace třídy Coefficients*. Online. Dostupné z:
https://docs.juce.com/master/structdsp_1_1IIR_1_1Coefficients.html. [cit. 2025-05-21].
- [29] *JUCE dokumentace třídy Component*. Online. Dostupné z:
<https://docs.juce.com/master/classComponent.html>. [cit. 2025-05-21].
- [30] *JUCE dokumentace třídy LookAndFeel*. Online. Dostupné z:
https://docs.juce.com/master/classLookAndFeel_V4.html. [cit. 2025-05-21].
- [31] *Font Instrument Sans*. Online. Dostupné z:
<https://fonts.google.com/specimen/Instrument+Sans>. [cit. 2025-05-23].
- [32] MURTHY, Akash. *xy-pad-demo*. Online. GitHub. 2021. Dostupné z:
<https://github.com/Thrifleganger/xy-pad-demo.git>. [cit. 2025-05-23].
- [33] SCHIERMEYER, Charles. *SimpleEQ*. Online. GitHub. 2021. Dostupné z:
<https://github.com/matkatmusic/SimpleEQ>. [cit. 2025-05-24].
- [34] RUSSEL, Jordan. *Inno Setup*. Online. Dostupné z:
<https://jrsoftware.org/isinfo.php> [cit. 2025-05-30].

- [35] SCHMILOVITZ, Doron. On the definition of total harmonic distortion and its effect on measurement interpretation. Online. *IEEE Transactions on Power Delivery*. 2005, roč. 20, č. 1, s. 526-528. Dostupné z: 10.1109/TPWRD.2004.839744. [cit. 2025-05-25].

Seznam symbolů a zkratek

AAX	Avid Audio eXtension
B	délka bloku
CCP	Cyclic Convolution Property
DC	Direct Current - stejnosměrná složka
DFT	Diskrétní Fourierova Transformace
FDL	Frequency-domain Delay-Line
FFT	Fast Fourier Transform
FIFO	First In First Out
FIR	Finite Impulse Response
f_{vz}	vzorkovací kmitočet
HP	High Pass
IFFT	Inverse Fast Fourier Transform
IIR	Infinite Impulse Response
LP	Low Pass
LTI	Linear Time-Invariant system
OLA	Overlap-Add
OLS	Overlap-Save
THD	Total Harmonic Distortion
VST3	Virtual Studio Technology verze 3

Seznam příloh

A Zdrojové soubory MATLAB	47
B Soubory JUCE	48

A Zdrojové soubory MATLAB

V příloze je obsažen soubor hlavní aplikace `IRSim.mlapp` spolu s pomocnými soubory MATLAB funkcí.

Dále je v příloze soubor Audio Toolbox modulu `convolutionPlugin.m` s pomocnými soubory MATLAB funkcí.

Složka s naměřenými impulsními odezvami ve formátu `.mat` není z důvodu velkého objemu dat přiložena, je však dostupná z odkazu https://drive.google.com/file/d/1gJecihg033Z-g_m7Cr6Q60Jn7-EsTIRF/view?usp=sharing. Tato složka je nutná pro správnou funkci MATLAB implementací.

B Soubory JUCE

Přiložen je zdrojový kód projektu JUCE – hlavní částí jsou 2 dvojice `.cpp` a `.h` souborů, pomocné třídy jsou zvlášť ve složce `Components`. Následně je přiložen instalátor VST3 pluginu a standalone verze aplikace `Installer.exe`. Pro možnost manuální instalace je přiložen `AmpSimulator.exe` – standalone verze – a soubor VST3 modulu `AmpSimulator.vst3`. Dále jsou přiloženy soubory impulsních odezev, `.png` soubory použité v GUI pluginu a návod na instalaci `README.txt`.

```
/.....kořenový adresář přiloženého archivu
├── JUCE
│   ├── Assets.....složka obsahující obrázky v GUI
│   ├── Data.....složka obsahující impulsní odezvy
│   ├── Source ..... zdrojový kód C++
│   │   ├── PluginEditor.cpp
│   │   ├── PluginEditor.h
│   │   ├── PluginProcessor.cpp
│   │   ├── PluginProcessor.h
│   │   └── Components..... složka obsahující třídy komponent
│   ├── AmpSimulator.exe ..... standalone aplikace
│   ├── AmpSimulator.vst3.....soubor VST3 pluginu
│   ├── Installer.exe ..... instalátor aplikace
│   └── README.txt.....návod k instalaci
├── MATLAB
│   ├── MatlabApp.....složka obsahující funkce aplikace
│   │   └── IRSim.mlapp..... soubor Matlab aplikace
│   ├── MatlabRT ..... složka obsahující funkce AudioToolbox
│   │   └── convolutionPlugin.m.....soubor Matlab AudioToolbox pluginu
```